

THỰC TẾ VÀ SẢN XUẤT



TÓM TẮT

Đây là phần ít tài liệu học thuật nhất và tạo ra giá trị thật nhiều nhất. Ba phần trước dựng một hệ thống chạy tốt trên EuRoC; phần này trả lời câu hỏi vì sao cùng hệ thống ấy chết ở nhà khách hàng, và phải làm gì. Bốn chương: thế giới thực và những gì nó ném vào hệ thống (chương 19); ràng buộc tính toán, bộ nhớ và thời gian trên phần cứng thật (chương 20); con đường từ một nguyên mẫu tới hàng nghìn máy xuất xưởng (chương 21); và cuối cùng, tám chỗ mà tôi cho là còn khoảng trống thật nếu mục tiêu là đóng góp mới (chương 22). Nếu chỉ nhớ một điều: *biết mình đang sai quan trọng hơn không bao giờ sai* — một hệ có tỉ lệ lỗi 2% mà phát hiện được 90% số lần lỗi thì dùng được trong sản phẩm; một hệ có tỉ lệ lỗi 1% mà im lặng khi lỗi thì không.

Phần này khác ba phần trước ở một điểm căn bản: *nó không có tài liệu chuẩn*. Không có sách giáo khoa nào về việc đưa SLAM lên dây chuyền, và số bài báo về chủ đề này ít hơn số bài về một biến thể detector đặc trưng vài bậc độ lớn. Lý do có cấu trúc: kiến thức ở đây thuộc về các công ty, nó không tổng quát hoá sạch sẽ thành định lý, và nó khó công bố vì mỗi ràng buộc là riêng của một sản phẩm.

Hệ quả cho cách đọc: nội dung ở đây được viết ở mức *nguyên tắc và danh mục kiểm* chứ không ở mức công thức. Nó cũng là phần mà tôi phải cẩn thận nhất về nguồn — rất nhiều điều “ai cũng biết” trong công nghiệp không có nguồn nào kiểm được, và theo `learning-rules.md` mục 5 thì tôi phải nói rõ chỗ nào là suy luận của tôi. Tôi làm điều đó nhất quán trong cả bốn chương.

Chương 19 và 21 là hai chương xương sống. Chương 19 vì nó liệt kê những gì thế giới thật ném vào hệ thống, và mỗi mục trong danh sách ấy là một chế độ hỏng mà ba phần trước hoặc bỏ qua hoặc chỉ nhắc thoáng. Chương 21 vì nó là chương duy nhất trong cây viết cho mục tiêu mass production, và vì các quyết định nó bàn — dung sai cơ khí, quy trình hiệu chuẩn, giám sát fleet — là những quyết định khó đảo ngược nhất. Chương 20 là chương kỹ thuật thuần túy và có thể đọc lướt nếu bạn chưa chạm vào phần cứng nhúng. Chương 22 khép cây lại bằng một danh sách tám hướng, và nó là chương duy nhất chạm vào địa hạt nghiên cứu — với một cảnh báo rõ ràng rằng nó chưa đạt chuẩn của `research-rules.md` và cần được nâng lên thành một dự án khảo sát riêng trước khi dùng để chọn đề tài thật.

Bảng 1. Bản đồ Phần D: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
19	19-ben-vung-trong-thi	Vật động và masking; môi trường nghèo vân; thay đổi ánh sáng; motion blur; kính và gương; vân lặp lại; ngoài trời; suy giảm cảm biến; phát hiện thất bại và suy giảm có kiểm soát	● ● ●
20	20-toi-uu-hieu-nang-	Đa luồng và hàng đợi; xử lý độ trễ và mất khung; SIMD và lượng tử hoá; GPU/NPU; ngân sách bộ nhớ; bộ giải thừa tối ưu; tính tất định; điện năng và nhiệt	● ●
21	21-tu-prototype-len-	Hiệu chuẩn quy mô sản xuất; độ tin cậy và giám sát sức khỏe; kiểm thử và regression; ràng buộc sản phẩm; fleet analytics; OTA; quyền riêng tư	● ● ●
22	22-huong-nghien-cuu-	Tám hướng còn khoảng trống, với đánh giá mức chắc chắn và cách kiểm mỗi hướng có thật là khoảng trống không	● ●

Chương 19 và 21 là xương sống của phần này. Đọc 19 trước ngay cả khi bạn còn đang làm nguyên mẫu — danh mục các chế độ hỏng ở đó nên được dùng làm danh mục kiểm khi thiết kế, không phải khi gỡ lỗi. Chương 20 đọc khi bạn bắt đầu chạm vào phần cứng mục tiêu; trước đó nó là kiến thức không dùng tới. Chương 21 là chương nên đọc *trước khi chốt thiết kế cơ khí*, vì vài quyết định nó bàn — dung sai lắp ráp, chọn global shutter, chỗ đặt cảm biến nhiệt — không sửa được sau khi mở khuôn. Chương 22 đọc cuối, và đọc với thái độ hoài nghi: nó là danh sách các chỗ tôi

cho là còn trống, không phải kết quả của một khảo sát có hệ thống, và chương đó nói rõ điều này ngay từ đầu.

Bền vững trong thế giới thực (Robustness in the Real World)

PHỤ THUỘC A/01-hình-học-da-thi (giả thiết cảnh tĩnh — chương này là chỗ trả nợ cho nó); A/02-mô-hình-camera (mô hình quang trắc, rolling shutter); B/07-dac-trung-va-data-association (giới hạn của RANSAC); B/08-mô-hình-phan-du (indirect và direct hồng khác nhau); C/13-quan-ly-ban-do (thế giới thay đổi theo thời gian).

TÓM TẮT

Ba phần trước dựng một hệ thống chạy tốt trên EuRoC. Chương này liệt kê những gì thế giới thật ném vào nó, và mỗi mục trong danh sách là một giả thiết của Phần A bị vi phạm. Nội dung tổ chức theo bốn nhóm chứ không theo danh sách phẳng, vì mỗi nhóm đòi một loại xử lý khác nhau. Nhóm một — *cảnh không tĩnh*: vật động, và một trường hợp đặc biệt tệ là khi vật động chiếm đa số khung hình. Nhóm hai — *thông tin không đủ*: nghèo vân, tối, mờ do chuyển động. Nhóm ba — *thông tin sai lệch*: kính, gương, bề mặt phản chiếu, vân lặp lại; đây là nhóm nguy hiểm nhất vì nó hồng *im lặng*. Nhóm bốn — *cảm biến suy giảm*: bụi ống kính, lệch tiêu cự, hồng pixel, và trôi hiệu chuẩn. Mục cuối cùng, và là mục quan trọng nhất, bàn về thứ quyết định một hệ có dùng được trong sản phẩm hay không: *phát hiện thất bại và suy giảm có kiểm soát*. Nếu chỉ nhớ một điều: *biết mình đang sai quan trọng hơn không bao giờ sai* — và đó không phải một câu an ủi mà là một nguyên tắc thiết kế có hệ quả cụ thể ở mọi module.

KIẾN THỨC CẦN CÓ TRƯỚC

- **vật động (dynamic object)** — vật thể di chuyển trong cảnh — người, xe, cửa mở. Nó vi phạm giả thiết cảnh tĩnh mà toàn bộ A/01 dựa vào.
- **masking** — loại bỏ các vùng ảnh khỏi việc xử lý, thường dựa trên phân đoạn ngữ nghĩa (“đây là người”) hoặc trên kiểm tra hình học (“vùng này không nhất quán”).
- **motion blur** — nhòe do vật hoặc camera di chuyển trong thời gian phơi sáng. Nó làm gradient giảm, nên cả detector lẫn direct method đều suy giảm.
- **suy giảm có kiểm soát (graceful degradation)** — hành vi của hệ thống khi điều kiện xấu: thay vì hồng hoàn toàn, nó chuyển sang chế độ kém chính xác hơn nhưng vẫn hoạt động, và *báo* rằng nó đang ở chế độ đó.
- **phát hiện thất bại (failure detection)** — cơ chế để hệ thống tự nhận ra ước lượng của nó không còn đáng tin. Khác với phục hồi ở chỗ nó chạy *trước* và quyết định có cần phục hồi không.
- **hồng im lặng (silent failure)** — chế độ hồng mà hệ thống tiếp tục cho ra kết quả trông hợp lý nhưng sai, không có dấu hiệu nào. Nguy hiểm hơn hồng ồn ào nhiều bậc.

CÂU HỎI MỞ ĐẦU

- Q1. Một chiếc xe tải đi ngang chiếm 70% khung hình. RANSAC báo 85% inlier. Nêu chính xác cái gì vừa được ước lượng, và vì sao kết quả sai mà vẫn nhất quán.
- Q2. Direct method và indirect method đều suy giảm khi có vật động. Nêu vì sao direct suy giảm *nặng* hơn, và nói điều đó ảnh hưởng tới lựa chọn kiến trúc thế nào.

- Q3. Trong bốn nhóm vấn đề ở chương này, nhóm nào nguy hiểm nhất và vì sao? Câu trả lời không phải nhóm làm hệ thống hỏng nhiều nhất.
- Q4. Camera của bạn bị mờ một góc vì bụi bám sau sáu tháng. Nếu ba dấu hiệu đo được, và nói vì sao ATE không phải một trong ba dấu hiệu ấy.
- Q5. Bạn có ngân sách cho *một* tính năng: cải thiện độ chính xác 20%, hoặc thêm một cơ chế phát hiện thất bại. Chọn cái nào và biện minh bằng chi phí vận hành.

1.1 Đặt vấn đề

A/01 mục 7 liệt kê bốn giả thiết của hình học đa thị và nói rằng ba trong bốn sẽ được trả nợ ở các chương sau. Chương này là chỗ trả nợ, và nó trả nợ cho nhiều hơn thế — gần như mọi giả thiết tiện lợi của ba phần trước đều bị vi phạm ở đây.

Bài toán gốc phát biểu bằng một quan sát thực nghiệm mà ai triển khai đều gặp: *một hệ SLAM chạy tốt trên bộ dữ liệu chuẩn thường thất bại ở môi trường thật, và nó thất bại vì những lý do không có trong bộ dữ liệu chuẩn.*

Lý do cấu trúc, và đáng nói rõ vì nó giải thích cả sự thiếu tài liệu. Bộ dữ liệu chuẩn được thu để *đánh giá thuật toán*, nên chúng cố ý loại bỏ nhiễu loạn: EuRoC quay trong một phòng có motion capture, không có người đi lại; KITTI quay trên đường, nhưng người ta chọn các đoạn mà hệ thống chạy được. Điều đó *đúng* về mặt phương pháp — để so hai thuật toán, cần một môi trường kiểm soát — nhưng nó nghĩa là điểm số trên chúng không dự đoán được hành vi ngoài đời.

Ai gặp bài toán này? Chỉ những người triển khai. Đây là lý do một kỹ sư có sáu tháng kinh nghiệm triển khai thường biết nhiều hơn về chủ đề này so với một người đọc trăm bài báo — và là lý do chương này viết ở mức danh mục kiểm chứ không ở mức công thức.

Cách tổ chức của chương đáng giải thích. Tôi *không* liệt kê phẳng các vấn đề mà chia thành bốn nhóm theo *bản chất của cái sai*, vì mỗi nhóm đòi một loại xử lý khác nhau và nhằm nhóm là nhằm cách chữa:

Nhóm một — cảnh không tĩnh: dữ liệu đúng nhưng mô hình sai (mô hình giả định thế giới đứng yên).

Nhóm hai — thông tin không đủ: mô hình đúng nhưng dữ liệu nghèo. Cách chữa là thêm nguồn thông tin.

Nhóm ba — thông tin sai lệch: dữ liệu *trông* đúng nhưng không phải. Nguy hiểm nhất vì mọi cơ chế phát hiện dựa trên tính nhất quán đều thất bại.

Nhóm bốn — cảm biến suy giảm: mô hình đo không còn mô tả cảm biến. Đây là nhóm phát triển *chậm* theo thời gian, nên nó không xuất hiện trong bất kỳ thí nghiệm ngắn nào.

TRỰC GIÁC

Hình dung bạn đang đi trong một toà nhà và ghi nhớ đường đi bằng cách nhìn các mốc.

Nhóm một: có người đi ngang. Bạn ghi nhớ “có một người mặc áo đỏ ở đây”, rồi lát sau quay lại và người đó đã đi mất. Tệ hơn: nếu bạn đi cùng chiều với người đó một đoạn dài, bạn sẽ nghĩ mình đứng yên trong khi thực ra cả hai đang đi — vì so với người ấy thì bạn không nhúc nhích.

Nhóm hai: đèn tắt, hoặc bạn đi vào một hành lang trắng trơn không có gì để nhớ. Bạn *biết* mình không có thông tin, và bạn đi chậm lại, quờ tay tìm tường. Đây là loại vấn đề dễ chịu nhất, vì bạn biết mình đang gặp nó.

Nhóm ba: có một tấm gương lớn. Bạn nhìn thấy một hành lang tiếp tục ở phía trước và đi thẳng vào đó. Mọi thứ bạn thấy đều hợp lý, nhất quán, rõ ràng — cho tới khi bạn đâm vào gương. Đây là loại tệ nhất, vì *không có gì báo trước*.

Nhóm bốn: mắt bạn mờ dần theo năm tháng. Mỗi ngày nó chỉ tệ hơn hôm qua một chút, nên không ngày nào bạn nhận ra. Sau vài năm bạn nhìn nhầm nhiều thứ, và bạn vẫn tin vào mắt mình như cũ.

Bốn loại, bốn cách xử lý. Loại hai chữa bằng cách thêm nguồn thông tin khác — sờ tường, đếm bước. Loại một và bốn chữa bằng cách nhận ra và điều chỉnh. Loại ba thì *không chữa được bằng cách nhìn kỹ hơn* — cách duy nhất là biết trước rằng gương tồn tại, và nghi ngờ khi thấy một hành lang quá hoàn hảo.

Và có một điều chung cho cả bốn, quan trọng hơn mọi cách chữa: *một người biết mình đang không chắc thì đi chậm lại và sờ tường; một người không biết thì đi bình thường và ngã*.

1.2 Nhóm một: cảnh không tĩnh

A. Vì sao RANSAC không đủ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất.

RANSAC tìm *tập nhất quán lớn nhất* (B/07 mục 5B). Khi vật động chiếm thiểu số, tập nhất quán lớn nhất là cảnh tĩnh, và RANSAC làm đúng việc của nó.

Khi vật động chiếm *đa số* — một xe tải đi ngang, một đám đông, một bức tường của kho hàng đang được xe nâng dịch chuyển — tập nhất quán lớn nhất là *vật động*. RANSAC trung thành chọn nó làm inlier và cảnh tĩnh làm outlier.

Cái vừa được ước lượng là *tư thế tương đối của camera so với chiếc xe tải*. Kết quả sai so với thế giới, nhưng nó hoàn toàn nhất quán: mọi điểm trên xe tải khớp hoàn hảo với mô hình, phần dư nhỏ, số inlier cao. Không có gì trong đầu ra báo hiệu.

Đây là một ví dụ nữa của nguyên tắc ở A/05: số inlier đo mức nhất quán của dữ liệu với mô hình, không đo mức đúng đắn của mô hình.

B. Ba tầng xử lý

Tầng một — kiểm tra hình học. Với các điểm có lịch sử quan sát đủ dài, kiểm xem chúng có nhất quán với một điểm 3D tĩnh không. Một điểm trên vật động sẽ có phần dư tăng dần khi vật di chuyển. Rẻ, không cần mô hình ngữ nghĩa, và nó bắt được mọi loại vật động — kể cả những loại mà một bộ phân đoạn không biết. Giới hạn: cần thời gian, nên nó không bắt được vật động ngay lập tức.

Tầng hai — masking ngữ nghĩa. Dùng một mạng phân đoạn để loại các vùng thuộc các lớp *có thể* di chuyển — người, xe, động vật. DynaSLAM [1] làm điều này. Nhanh và chủ động, nhưng nó có hai giới hạn đáng nêu: nó chỉ biết các lớp đã huấn luyện, và nó loại cả những vật *đang đứng yên* thuộc lớp đó — một chiếc xe đỗ là một mốc tĩnh rất tốt mà masking ngữ nghĩa sẽ vứt đi.

Cách kết hợp đúng: dùng ngữ nghĩa để *hạ trọng số*, không để loại cứng; rồi để kiểm tra hình học quyết định. Một chiếc xe đỗ sẽ chứng minh mình tĩnh qua vài khung và lấy lại trọng số.

Tầng ba — nhận ra khi vật động chiếm đa số. Đây là tầng hay bị bỏ và là tầng quan trọng nhất. Khi tầng một và hai cùng báo rằng phần lớn khung hình là động, hành vi đúng *không* phải cố ước lượng từ phần còn lại — đó là *từ chối* và dựa vào cảm biến khác (C/15) cho tới khi tình hình qua đi. Đây là nguyên tắc “biết từ chối” của B/10 áp dụng ở một chỗ khác.

C. Direct suy giảm nặng hơn indirect

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

Với indirect, một vật động làm mất vài chục điểm rời rạc — các điểm trên vật đó trở thành outlier và bị M-estimator hạ trọng số. Hệ thống mất một phần thông tin.

Với direct, một vật động làm hỏng cường độ của *cả một vùng liên thông*. Vì phần dư quang trắc được tính trên mọi pixel có gradient, và vì các pixel trong vùng ấy *tương quan mạnh* với nhau (chúng cùng sai theo cùng một cách), nên đóng góp của vùng ấy vào hàm mục tiêu lớn hơn nhiều so với tỉ lệ diện tích của nó. M-estimator xử lý outlier độc lập tốt hơn nhiều so với outlier tương quan thành khối.

Hệ quả kiến trúc: trong môi trường có nhiều vật động — nhà xưởng có người, đường phố — indirect có lợi thế thật. Đây là một trong những tiêu chí cụ thể để chọn giữa hai họ ở B/08, và nó không xuất hiện trong các so sánh trên bộ dữ liệu tĩnh.

1.3 Nhóm hai: thông tin không đủ

Nhóm này dễ chịu nhất vì hệ thống *biết* nó đang gặp vấn đề — số đặc trưng giảm, phần dư tăng, hiệp phương sai nở ra.

Nghèo nàn: hành lang trắng, tường trơn, sương mù. Cách xử lý theo thứ tự: chuyển sang đặc trưng đường thẳng và mặt phẳng (B/08 mục 4B), vốn tồn tại ngay cả khi không có góc; hoặc chuyển sang direct, vốn khai thác được gradient yếu; hoặc dựa vào cảm biến khác cho tới khi qua.

Ánh sáng thay đổi: đèn bật tắt, đi từ trong nhà ra ngoài trời, auto-exposure phản ứng. Với indirect, descriptor bền một phần. Với direct, đây là vấn đề nghiêm trọng và cách chữa là mô hình quang trắc (A/02 mục 5B) cộng hai tham số affine mỗi khung.

Motion blur: gradient giảm nên cả hai họ đều suy giảm. Cách chữa đúng thuộc về phần cứng — giảm thời gian phơi sáng, tăng gain, dùng cảm biến nhạy hơn — và đó là một quyết định ở D/21. Cách chữa phần mềm là dùng IMU để dự đoán và thu hẹp vùng tìm kiếm (B/07 mục 5C), vốn giúp nhưng không giải quyết được việc thông tin đã mất.

Điểm chung của cả ba, và là lý do nhóm này dễ hơn: *có tín hiệu cảnh báo*. Số đặc trưng, độ tương phản trung bình, và mức gradient đều đo được rõ, và chúng giảm *trước* khi hệ thống hỏng. Một hệ có giám sát các đại lượng này sẽ biết trước vài trăm mili giây — đủ để chuyển chế độ hoặc để báo.

1.4 Nhóm ba: thông tin sai lệch

Đây là câu trả lời cho câu hỏi mở đầu thứ ba: nhóm này nguy hiểm nhất, và không phải vì nó làm hệ thống hỏng nhiều nhất mà vì nó hỏng *im lặng*.

A. Kính, gương, bề mặt phản chiếu

Trong nhóm một và hai, hệ thống hoặc thấy phần dư lớn hoặc thấy ít dữ liệu — có tín hiệu. Ở đây, mọi thứ *trông hoàn hảo*: các đặc trưng trong gương ổn định, khớp tốt qua nhiều khung, triangulate ra các điểm 3D nhất quán. Chúng chỉ ở sai chỗ — ở phía sau mặt gương, tại vị trí ảnh phản chiếu.

Với cửa kính còn tệ hơn: hệ thống thấy cả cảnh phía sau kính (đúng về mặt hình học, vì ánh sáng đi thẳng qua) lẫn ảnh phản chiếu trên mặt kính (sai), trộn vào nhau.

Vì sao khó chữa: mọi cơ chế phát hiện trong cây này dựa trên *tính nhất quán* — phần dư, số inlier, kiểm chéo giữa các nguồn. Ảnh phản chiếu nhất quán với chính nó qua nhiều khung, nên không cơ chế nào bắt được. Đây là giới hạn cấu trúc chứ không phải thiếu sót cài đặt.

Ba hướng giảm thiểu, không cái nào đầy đủ. *Ngữ nghĩa*: một bộ phân đoạn có lớp “kính” và “gương” — hoạt động khi được huấn luyện đúng, và các bề mặt ấy khó phân đoạn chính vì chúng trong suốt. *Nhất quán đa cảm biến*: LiDAR cũng bị lừa bởi gương nhưng theo cách khác, nên bất đồng giữa hai cảm biến là một tín hiệu. *Suy luận hình học cấp cao*: ảnh phản chiếu có cấu trúc hình học đặc biệt — nó là phản xạ của cảnh thật qua một mặt phẳng — và về nguyên tắc một hệ có thể nhận ra điều đó qua nhiều góc nhìn. Tôi không biết một hệ SLAM thực dụng nào làm điều này, và nó là một hướng chưa được khai thác (*tôi suy ra từ lập luận hình học, chưa đổi chiều nguồn*).

B. Vân lặp lại

Đã bàn ở B/07 mục 3B và B/11 mục 5B, và nó thuộc nhóm này vì cùng lý do: một khớp sai giữa hai cánh cửa giống nhau *nhất quán* với chính nó.

Điều đáng nhắc ở đây là một biểu hiện đặc biệt trong sản phẩm: các môi trường lặp lại có hệ thống — kho hàng, bãi đỗ xe, hành lang chung cư — là đúng những môi trường mà robot thương mại hoạt động. Nên đây không phải một trường hợp biên mà là môi trường mặc định của một phần lớn thị trường.

C. Vì sao nhóm này quyết định thiết kế

Ba nhóm kia có thể xử lý bằng cách cải thiện thuật toán. Nhóm này thì không — nó đòi *thay đổi kiến trúc hệ thống*, theo hai hướng.

Một — đa dạng hoá cảm biến. Không phải để chính xác hơn mà để có *bất đồng*. Hai cảm biến bị lừa theo hai cách khác nhau sẽ bất đồng, và bất đồng là tín hiệu duy nhất có sẵn trong nhóm này. Đây là một lập luận cho hợp nhất cảm biến khác với lập luận ở C/15, và có lẽ mạnh hơn.

Hai — giới hạn hậu quả. Nếu không phát hiện được, hãy đảm bảo hậu quả bị chặn: đừng để một loop closure đơn lẻ phá bản đồ (B/11 mục 5), đừng để một relocalization đơn lẻ được tin ngay (B/12 mục 4B), và giữ khả năng quay lại một trạng thái trước đó.

1.5 Nhóm bốn: cảm biến suy giảm

Đây là nhóm mà không thí nghiệm ngắn nào phát hiện được, vì nó phát triển theo tháng và năm. Đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Bụi và vết bẩn trên ống kính. Một vùng ảnh mất tương phản. Ba dấu hiệu đo được: mật độ đặc trưng theo vùng giảm không đều (một góc ảnh sứt trong khi các góc khác bình thường); độ tương phản cục bộ giảm ở cùng vùng đó; và phân bố inlier lệch khỏi vùng ấy. Cả ba đo được mà không cần chân trị, và cả ba nên là một phần của giám sát sức khoẻ ở D/21.

Vì sao ATE *không* phải một dấu hiệu: một vùng ảnh mất đặc trưng không làm hệ thống sai ngay — các đặc trưng còn lại vẫn đủ, và ATE có thể không đổi trong nhiều tháng. Nó chỉ tệ đi khi kết hợp với một điều kiện khó khác, và lúc đó bạn không biết nguyên nhân là bụi. Đây là một ví dụ chung của nguyên tắc: *đo nguyên nhân, không chỉ đo hậu quả*.

Lệch tiêu cự và trôi cơ khí. Ống kính xoay dần do rung; keo lão hoá; khung stereo biến dạng theo nhiệt. Biểu hiện: phần dư có cấu trúc không gian (A/02 mục 4A) hoặc tham số hiệu chuẩn trực tuyến trôi đơn điệu (C/14 mục 4B).

Hỏng pixel và suy giảm cảm biến. Pixel chết tạo ra các đặc trưng giả *đứng yên trong ảnh*. Đây là một chế độ hỏng tinh vi: các đặc trưng ấy ổn định tuyệt đối qua mọi khung, nên chúng trông rất đáng tin — và chúng nói rằng camera không di chuyển. Phát hiện: một đặc trưng có cùng toạ độ pixel qua hàng nghìn khung trong khi hệ thống đang di chuyển là một pixel hỏng, không phải một landmark.

CẦN NHỚ

Do nguyên nhân, không chỉ do hậu quả. Đây là nguyên tắc nổi chương này với D/21.

ATE là hậu quả. Nó tổng hợp mọi nguyên nhân thành một con số, và khi nó xấu đi bạn không biết vì sao. Tệ hơn, nó có thể *không* xấu đi trong nhiều tháng trong khi hệ thống đang suy giảm dần — vì các cơ chế bù trừ vẫn còn dư địa.

Các đại lượng đo nguyên nhân thì rẻ và nhiều thông tin hơn. Danh sách tối thiểu nên giám sát liên tục trong sản phẩm:

Nhóm một: tỉ lệ điểm bị loại vì không nhất quán hình học; diện tích vùng bị mask.

Nhóm hai: số đặc trưng, độ tương phản trung bình, mức gradient trung bình, thời gian phơi sáng.

Nhóm ba: bất đồng giữa các cảm biến; số loop closure bị từ chối ở từng tầng.

Nhóm bốn: mật độ đặc trưng *theo vùng ảnh* (không phải tổng); xu hướng dài hạn của tham số hiệu chuẩn trực tuyến; số đặc trưng có toạ độ pixel không đổi.

Mỗi đại lượng tốn vài byte mỗi khung và có thể tổng hợp thành thống kê hàng ngày. Chúng là thứ biến một báo cáo lỗi “robot bị lạc” thành một chẩn đoán “camera trái bắn từ ba tuần trước”.

1.6 Phát hiện thất bại và suy giảm có kiểm soát

Đây là mục quan trọng nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ năm.

A. Lập luận chi phí

So sánh hai hệ thống trong một đội hình một nghìn robot.

Hệ A: tỉ lệ lỗi 1% số ca, và khi lỗi nó im lặng — tiếp tục báo tư thế sai. Mười ca mỗi ngày, mỗi ca robot đi sai chỗ, có thể va chạm, và cần người tới xử lý. Chi phí: mười lần can thiệp, cộng rủi ro an toàn, cộng việc khách hàng mất niềm tin.

Hệ B: tỉ lệ lỗi 2%, nhưng nó phát hiện được 90% số ca. Hai mươi ca mỗi ngày, trong đó mười tám ca robot *dừng lại và báo*, và hai ca im lặng. Chi phí: mười tám lần gián đoạn ngắn (robot dừng, tự phục hồi hoặc chờ) cộng hai lần can thiệp.

Hệ B tốt hơn rõ rệt dù tỉ lệ lỗi gấp đôi, vì *một lỗi được phát hiện rẻ hơn một lỗi im lặng nhiều bậc*. Đây là lập luận cho việc chọn phát hiện thất bại thay vì cải thiện độ chính xác khi chỉ có ngân sách cho một cái.

Có một lập luận thứ hai, và nó mạnh hơn về dài hạn: *một lỗi được phát hiện là một lỗi được ghi lại*. Hệ B tạo ra dữ liệu — mười tám bản ghi mỗi ngày về việc nó thất bại ở đâu và trong điều kiện nào. Sau một tháng đó là năm trăm mẫu để phân tích. Hệ A tạo ra các báo cáo mơ hồ từ khách hàng. Đây là vòng lặp cải tiến ở D/21, và nó chỉ khởi động được nếu hệ thống biết mình sai.

B. Bốn tầng phát hiện

Tầng một — các đại lượng tức thời, mỗi khung: số inlier, tỉ lệ tracking, phần dư trung bình. Rẻ nhất, phát hiện sớm nhất, nhiều báo động giả nhất.

Tầng hai — các đại lượng thống kê, trên một cửa sổ: xu hướng của phần dư, phân bố không gian của inlier, số điều kiện của Λ . Bắt được các suy giảm chậm mà tầng một bỏ qua.

Tầng ba — nhất quán chéo giữa các nguồn: bắt đồng giữa thị giác và IMU, giữa thị giác và odometry. Đây là tầng duy nhất có cơ hội bắt được nhóm ba, và nó chỉ tồn tại nếu có nhiều cảm biến.

Tầng bốn — kiểm tính hợp lý vật lý: robot có thể đi nhanh thế này không? Tư thế có nhảy một quãng không thể trong một khung không? Rẻ, và nó bắt được các thất bại thảm họa mà ba tầng kia có thể bỏ qua vì chúng dựa trên các đại lượng nội tại vốn cũng bị ảnh hưởng.

C. Suy giảm có kiểm soát

Phát hiện xong thì làm gì. Nguyên tắc: *chuyển sang chế độ kém chính xác hơn nhưng vẫn hoạt động, và báo rằng mình đang ở chế độ đó.*

Một thang chế độ hợp lý, từ tốt tới xấu: SLAM đầy đủ với bản đồ; SLAM không đóng vòng (khi loop closure không đáng tin); odometry thị giác-quán tính không bản đồ; odometry quán tính-bánh xe (khi thị giác chết); và cuối cùng là dừng và báo.

Hai yêu cầu về kiến trúc, và cả hai phải được thiết kế từ đầu. Một — *chuyển chế độ phải mượt, không có bước nhảy trong đầu ra, vì bộ điều khiển phía trên không chịu được.* Hai — *chế độ hiện tại phải là một phần của đầu ra, cùng với tư thế và hiệp phương sai.* Một module phía trên nhận “tư thế, hiệp phương sai, chế độ” hành xử khác hẳn một module chỉ nhận tư thế — nó biết khi nào nên thận trọng.

Yêu cầu thứ hai nghe hiển nhiên và nó bị bỏ sót thường xuyên, vì trong nghiên cứu đầu ra của SLAM là một quỹ đạo để so với chân trị, không phải một tín hiệu cho một hệ thống khác dùng.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DynaSLAM [1]	Masking ngữ nghĩa + kiểm tra hình học cho vật động	Mục 2B; chú ý giới hạn của masking cứng
DSO [2]	Mô hình quang trắc + hai tham số affine mỗi khung	Mục 3 — xử lý thay đổi ánh sáng cho direct
ORB-SLAM3 [3]	Atlas cho phục hồi mềm khi mất bám	Mục 6C — một dạng suy giảm có kiểm soát
OpenLORIS [4]	Bộ dữ liệu có người đi lại, ánh sáng đổi, nhiều phiên	Một trong ít bộ dữ liệu chạm vào nội dung chương này
Kimera [5]	Ngữ nghĩa trong đồ thị; hạ trọng số theo lớp đối tượng	Hướng đúng cho mục 2B — hạ trọng số thay vì loại cứng

MINI PROJECT SLAM • Dựng một bộ thử nghiệm nghịch cảnh, rồi đo tỉ lệ phát hiện thất bại • 4 ngày

Mục tiêu	chuyển từ “hệ của tôi chạy tốt” sang “hệ của tôi thất bại trong $X\%$ các tình huống này, và nó <i>biết</i> trong $Y\%$ số lần thất bại”.
Dữ liệu	tự thu, và đây là phần chính của bài: dựng một bộ mười chuỗi nghịch cảnh, mỗi chuỗi nhắm vào một mục trong bốn nhóm — người đi ngang chiếm phần lớn khung hình, hành lang trắng, đèn tắt đột ngột, chuyển động nhanh gây mờ, một tấm gương lớn, một hành lang có các cửa giống nhau, một ống kính bị che một phần bằng băng dính mờ (mô phỏng bụi), và ba chuỗi tổ hợp.
Các bước	chạy hệ thống trên cả mười chuỗi; với mỗi chuỗi ghi lại: có thất bại không (so với chân trị thô hoặc quan sát bằng mắt), và hệ thống có <i>báo</i> rằng nó đang gặp vấn đề không; cài bốn tầng phát hiện ở mục 6B và đo tỉ lệ phát hiện cùng tỉ lệ báo động giả của từng

- tầng; cuối cùng cài một thang chế độ theo mục 6C và đo xem hệ thống có chuyển chế độ đúng lúc không.
- Xong khi** bạn có hai con số cho hệ thống của mình: *tỉ lệ thất bại* trên bộ nghịch cảnh, và *tỉ lệ phát hiện* trong số các lần thất bại. Con số thứ hai là con số mà gần như không ai công bố và nó quan trọng hơn con số thứ nhất. Bạn cũng có một bảng bốn tầng phát hiện với chi phí và hiệu quả của từng tầng — dùng nó để quyết định tầng nào đáng cài trước.
- Nối với** [D/20-toi-uu-hieu-nang-va-he-thong](#) cho chi phí tính toán của các tầng giám sát; [D/21-tu-prototype-len-mass-production](#) biến bộ nghịch cảnh này thành regression test và biến các đại lượng giám sát thành fleet analytics.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1.** Cái vừa được ước lượng là *tư thế tương đối của camera so với chiếc xe tải*, không phải so với thế giới. RANSAC tìm tập nhất quán *lớn nhất*, và khi vật động chiếm đa số thì tập ấy là vật động — nó trung thành chọn xe tải làm inlier và cảnh tĩnh làm outlier. Kết quả sai nhưng *hoàn toàn nhất quán*: mọi điểm trên xe khớp hoàn hảo với mô hình, phần dư nhỏ, số inlier cao, không có gì trong đầu ra báo hiệu. Đây lại là nguyên tắc ở A/05: số inlier đo mức nhất quán của dữ liệu với mô hình, không đo mức đúng đắn của mô hình.
- A2.** Vì một vật động làm hỏng cường độ của *cả một vùng liên thông*, và các pixel trong vùng ấy *tương quan mạnh* — chúng cùng sai theo cùng một cách. M-estimator xử lý outlier độc lập tốt hơn nhiều so với outlier tương quan thành khối, nên đóng góp sai của vùng ấy vào hàm mục tiêu lớn hơn tỉ lệ diện tích của nó. Với indirect, vật động chỉ làm mất vài chục điểm rời rạc và chúng bị hạ trọng số độc lập. Hệ quả kiến trúc: trong môi trường nhiều vật động — nhà xưởng có người, đường phố — indirect có lợi thế thật, và đây là một tiêu chí chọn giữa hai họ ở B/08 không xuất hiện trong các so sánh trên bộ dữ liệu tĩnh.
- A3.** *Nhóm ba — thông tin sai lệch* (kính, gương, vân lặp lại). Không phải vì nó làm hỏng nhiều nhất mà vì nó hỏng *im lặng*: mọi cơ chế phát hiện trong cây này dựa trên tính nhất quán, và ảnh phản chiếu nhất quán với chính nó qua nhiều khung nên không cơ chế nào bắt được. Đây là giới hạn cấu trúc chứ không phải thiếu sót cài đặt. Hệ quả: ba nhóm kia xử lý bằng cách cải thiện thuật toán, nhóm này đòi *thay đổi kiến trúc* — đa dạng hoá cảm biến để có *bất đồng* (tín hiệu duy nhất có sẵn), và giới hạn hậu quả của một lần sai đơn lẻ.
- A4.** Ba dấu hiệu: (1) *mật độ đặc trưng theo vùng ảnh* giảm không đều — một góc sứt trong khi các góc khác bình thường; (2) *độ tương phản cục bộ* giảm ở cùng vùng đó; (3) *phân bố inlier lệch khỏi vùng* ấy. Cả ba đo được không cần chân trị. ATE không phải dấu hiệu vì một vùng mất đặc trưng không làm hệ sai ngay — các đặc trưng còn lại vẫn đủ, và ATE có thể không đổi trong nhiều tháng; nó chỉ tệ đi khi kết hợp với một điều kiện khó khác, và lúc đó bạn không biết nguyên nhân là bại. Nguyên tắc chung: *đo nguyên nhân, không chỉ đo hậu quả*.
- A5.** Chọn *phát hiện thất bại*. Lập luận chi phí trên một nghìn robot: hệ A lỗi 1% và im lặng cho mười ca mỗi ngày, mỗi ca cần người tới xử lý cộng rủi ro an toàn; hệ B lỗi 2% nhưng phát hiện 90% cho mười tám lần gián đoạn ngắn (robot dừng và báo) cộng chỉ hai lần can thiệp. Hệ B tốt hơn rõ rệt dù tỉ lệ lỗi gấp đôi. Có một lập luận thứ hai mạnh hơn về dài hạn: *một lỗi được phát hiện là một lỗi được ghi lại* — hệ B tạo ra năm trăm mẫu mỗi tháng để phân tích, hệ A tạo ra các báo cáo mơ hồ từ khách hàng. Vòng lặp cải tiến ở [D/21](#) chỉ khởi động được nếu hệ thống biết mình sai.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đi trong một toà nhà lạ và nhớ đường bằng cách nhìn các mốc. Có bốn kiểu chuyện có thể xảy ra, và chúng khác nhau về bản chất chứ không chỉ về mức độ.

Kiểu một: có người đi ngang. Bạn ghi nhớ “có anh áo đỏ đứng đây” rồi lát sau quay lại, anh ta đã đi mất. Tệ hơn: nếu bạn đi cùng chiều với anh ta một đoạn dài, bạn sẽ tưởng mình đứng yên — vì so với anh ta thì bạn không nhúc nhích.

Kiểu hai: đèn tắt, hoặc bạn vào một hành lang trắng trơn không có gì để nhớ. Chuyện này khó chịu nhưng dễ chịu nhất trong bốn kiểu, vì *bạn biết* mình đang không có thông tin. Bạn tự khắc đi chậm lại và quờ tay tìm tường.

Kiểu ba: có một tấm gương lớn ở cuối hành lang. Bạn thấy hành lang tiếp tục ở phía trước và đi thẳng vào. Mọi thứ bạn thấy đều hợp lý, rõ ràng, nhất quán — cho tới khi bạn đâm vào gương. Đây là kiểu tệ nhất, vì không có gì báo trước và vì *nhìn kỹ hơn cũng không giúp gì*.

Kiểu bốn: mắt bạn mờ dần theo năm tháng. Mỗi ngày chỉ tệ hơn hôm qua một chút nên không ngày nào bạn nhận ra. Sau vài năm bạn nhìn nhầm nhiều thứ, và bạn vẫn tin vào mắt mình y như cũ.

Bốn kiểu, bốn cách xử lý. Kiểu hai chữa bằng cách dùng giác quan khác — sờ tường, đếm bước. Kiểu một và bốn chữa bằng cách để ý và điều chỉnh. Kiểu ba thì chỉ có một cách: *biết trước rằng gương tồn tại*, và thấy nghi khi một hành lang trông quá hoàn hảo.

Nhưng điều quan trọng nhất thì chung cho cả bốn, và nó không phải một mẹo kỹ thuật. Người biết mình đang không chắc thì đi chậm lại và sờ tường; người không biết thì đi bình thường và ngã. Hai người ấy có thể có cùng thị lực — nhưng một người về được nhà.

Chỗ không rút gọn được: chỗ không rút gọn được là tấm gương. Mọi cách nhận ra sai sót trong chương này đều dựa trên việc “có gì đó không khớp”. Ảnh trong gương khớp hoàn hảo với chính nó, khớp qua nhiều lần nhìn, khớp từ nhiều góc. Không có gì không khớp cả. Cách duy nhất là dùng một giác quan khác — hoặc biết trước.

Một câu: Biết mình đang sai quan trọng hơn không bao giờ sai — vì cái thứ nhất đo được và sửa được, cái thứ hai thì không tồn tại.

1.7 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *phát hiện thất bại có giá trị cao hơn cải thiện độ chính xác* — dựa trên một lập luận chi phí với các con số *tôi tự đặt* để minh họa. Nó bị bác bỏ nếu trong một triển khai thật, chi phí của một lần gián đoạn được phát hiện xấp xỉ chi phí của một lần lỗi im lặng — điều có thể đúng nếu việc robot dừng lại cũng đòi hỏi can thiệp của con người. Với robot có khả năng tự phục hồi thì mệnh đề vững; với robot mà mọi lần dừng đều cần người thì nó yếu đi nhiều. Đây là chỗ tôi phát biểu dựa trên cấu trúc chứ không dựa trên số liệu fleet (*tôi suy ra, chuta đối chiếu nguồn*).

Mệnh đề thứ hai — *nhóm ba nguy hiểm nhất* — là một xếp hạng theo mức khó phát hiện chứ không theo tần suất. Nó bị bác bỏ nếu trong thống kê thật, các thất bại do kính và gương hiểm tới mức không đáng kể so với các thất bại do nghèo vãn. Điều đó *có thể* đúng tùy môi trường — một nhà kho không có gương, một văn phòng hiện đại thì đầy kính. Cách kiểm là thu thống kê nguyên nhân từ fleet, tức đúng việc mà D/21 đề nghị.

Mệnh đề thứ ba — *direct suy giảm nặng hơn indirect với vật động* — suy từ lập luận về tương quan của outlier (cửa a). Nó kiểm được bằng thí nghiệm: chạy cả hai họ trên cùng chuỗi có vật động chiếm các tỉ lệ diện tích khác nhau, và xem đường cong suy giảm. Tôi *chưa* chạy thí nghiệm đó, và nó là một mục trong 99-chua-biet.md.

Mệnh đề thứ tư — *suy luận hình học cấp cao có thể phát hiện gương* — là một phỏng đoán của tôi dựa trên việc ảnh phản chiếu có cấu trúc đặc biệt. Tôi không biết một hệ thực dụng nào làm điều này, và tôi cũng chưa khảo sát xem có ai đã thử và thất bại hay không. Đây là chỗ có rủi ro sai cao nhất trong chương.

1.8 Kết nối

Nói lên trên. A/01-hình-hoc-da-thi mục 7 nêu bốn giả thiết ngầm; chương này là chỗ trả nợ cho giả thiết cảnh tĩnh. A/05-observability-gauge-va-suy-bien cho nguyên tắc chung mà mục 2A là một ứng dụng: số inlier không đo mức đúng đắn của mô hình. A/06-bat-dinh-va-lan-truyen-sai-so cho khái niệm hồng im lặng, mà nhóm ba là biểu hiện

cực đoan nhất. B/08-mo-hình-phan-du là chỗ mục 2C cho thêm một tiêu chí chọn giữa indirect và direct.

Nổi ngang. C/13-quan-ly-ban-do mục 4C bàn một dạng khác của cùng vấn đề: thế giới thay đổi theo tháng thay vì theo giây. C/14-hieu-chuan-nhu-mot-nhanh-cua-slam mục 4B cho cơ chế phát hiện trôi hiệu chuẩn, tức nhóm bốn. C/15-hop-nhat-da-cam-bien cho một lập luận về đa cảm biến; mục 4C ở đây cho một lập luận khác và có lẽ mạnh hơn — đa dạng hoá để có bất đồng. C/17-hoc-may-trong-slam cho công cụ ngữ nghĩa dùng ở mục 2B.

Nổi xuống dưới. D/20-toi-uu-hieu-nang-va-he-thong cho ngân sách tính toán của các tầng giám sát ở mục 6B — chúng không miễn phí. D/21-tu-prototype-len-mass-production là chỗ chương này trở thành quy trình: bộ nghịch cảnh thành regression test, các đại lượng giám sát thành fleet analytics, và tỉ lệ phát hiện thất bại thành một chỉ tiêu nghiệm thu. D/22-huong-nghien-cuu-con-trong lấy *dự đoán thất bại* — dự đoán trước khi mất bám thay vì phục hồi sau — làm hướng số bốn, và mục 6 của chương này là lập luận vì sao nó đáng giá.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao RANSAC chọn vật động khi nó chiếm đa số, và cái gì thực sự được ước lượng lúc đó?
2. Nêu ba tầng xử lý vật động, và nói vì sao masking ngữ nghĩa nên hạ trọng số thay vì loại cứng.
3. Vì sao direct suy giảm nặng hơn indirect khi có vật động? Trả lời bằng ngôn ngữ tương quan của outlier.
4. Nhóm nào trong bốn nhóm nguy hiểm nhất và vì sao? Nêu hai hướng thay đổi kiến trúc mà nó đòi hỏi.
5. Nêu ba dấu hiệu của bụi ống kính, và nói vì sao ATE không phải một trong ba.
6. Nêu bốn tầng phát hiện thất bại, và nói tầng nào là tầng duy nhất có cơ hội bắt được nhóm ba.
7. Trình bày lập luận chi phí cho việc ưu tiên phát hiện thất bại hơn độ chính xác, và nêu điều kiện mà lập luận đó yếu đi.

Nguồn

- [1] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [2] J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [6] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [7] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [8] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [9] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.
- [10] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [13] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.
- [14] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.

- [15] J. Maye, P. Furgale, and R. Siegwart, "Self-Supervised Calibration for Robotic Systems," in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [16] T. Schneider, M. Li, C. Cadena, et al., "Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation," in *IEEE Sensors Journal*, mức tin B, 2019.

Tối ưu hiệu năng và hệ thống (Performance and Systems Engineering)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (cấu trúc thừa — nguồn của phần lớn chi phí và của phần lớn cơ hội tối ưu); B/09-paradigm-uoc-luong-back-end (lựa chọn kiến trúc quyết định hồ sơ chi phí); C/13-quan-ly-ban-do mục 4A (kiến trúc nhiều luồng — chương này bàn chi tiết kỹ thuật của nó).

TÓM TẮT

Chương này về khoảng cách giữa một thuật toán đúng và một hệ thống chạy được trên phần cứng thật, trong thời gian thật, với ngân sách điện thật. Đó là khoảng cách mà bài báo không mô tả và nó chiếm phần lớn thời gian kỹ thuật của một dự án. Nội dung có bốn phần. Một: thời gian — và luận điểm trung tâm là *phân vị chứ không phải trung bình*, cùng cách xử lý độ trễ và mất khung mà không làm hỏng tính đúng đắn. Hai: kiến trúc đa luồng, nơi ba nhịp độ của C/13 gặp thực tế của khoá, hàng đợi và cạnh tranh dữ liệu — và nơi *tính tất định* nổi lên như một yêu cầu không thể thương lượng. Ba: ngân sách tính toán và bộ nhớ — SIMD, lượng tử hoá, bộ giải thừa, và các quyết định về nơi đặt mỗi phần của pipeline. Bốn: một vòng lặp vật lý mà gần như mọi tài liệu bỏ qua — điện năng, nhiệt, và việc nhiệt làm trôi hiệu chuẩn. Nếu chỉ nhớ một điều: *một hệ nhanh trung bình nhưng thỉnh thoảng chậm là một hệ không dùng được trong vòng điều khiển* — và mọi quyết định trong chương này xoay quanh việc làm cho trường hợp xấu nhất có giới hạn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phân vị (percentile)** — giá trị mà một tỉ lệ nhất định các mẫu nằm dưới nó. Phân vị 99 của thời gian xử lý là ngưỡng mà 99% số khung xử lý xong dưới đó.
- **jitter** — độ biến thiên của thời gian xử lý giữa các khung. Một hệ có jitter lớn khó dùng trong vòng điều khiển ngay cả khi trung bình thấp.
- **tính tất định (determinism)** — cùng đầu vào cho cùng đầu ra, chính xác tới từng bit, qua nhiều lần chạy. Điều kiện để tái hiện một lỗi từ hiện trường.
- **SIMD** — *single instruction multiple data* — chỉ thị xử lý nhiều phần tử cùng lúc. NEON trên ARM, AVX trên x86.
- **allocation-free hot loop** — vòng lặp chính không cấp phát bộ nhớ động. Cấp phát có thời gian không xác định và có thể gây phân mảnh, nên nó là nguồn jitter.
- **thermal throttling** — việc giảm tần số xử lý khi nhiệt độ vượt ngưỡng. Trên thiết bị nhúng không có quạt, đây là ràng buộc thật chứ không phải trường hợp biên.
- **ISP** — *image signal processor* — khối phần cứng xử lý ảnh thô từ cảm biến. Các thao tác của nó (auto-exposure, gamma) ảnh hưởng tới SLAM và thường không điều khiển được đầy đủ.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ của bạn xử lý trung bình 18 ms mỗi khung, dư dả cho 30 Hz. Nó vẫn không dùng được trong vòng điều khiển. Nêu đại lượng bạn chưa đo, và giải thích vì sao nó quyết định.
- Q2. Bạn tối ưu và giảm thời gian trung bình từ 18 ms xuống 12 ms. Nêu một trường hợp mà thay đổi đó làm hệ thống *tệ đi*.

- Q3. Đồng nghiệp đề nghị dùng số thực độ chính xác đơn thay vì kép để tăng tốc gấp đôi. Nên điều kiện để làm được điều đó một cách an toàn, và chỉ ra chương nào trong cây cung cấp điều kiện ấy.
- Q4. Một lỗi xảy ra ở hiện trường và bạn có log đầy đủ. Bạn chạy lại trên máy của mình và không tái hiện được. Nên nguyên nhân có khả năng nhất, và nói nó lẽ ra phải được ngăn từ lúc nào.
- Q5. Thiết bị chạy tốt trong mười phút rồi bắt đầu kém chính xác dần, và nó hồi phục sau khi nghỉ. Nên hai cơ chế hoàn toàn khác nhau có thể gây ra điều đó.

2.1 Đặt vấn đề

Ba phần trước nói về thuật toán đúng. Chương này về khoảng cách giữa một thuật toán đúng và một hệ thống chạy được, và khoảng cách ấy lớn hơn nhiều so với vẻ ngoài.

Bài toán gốc: thuật toán chạy trên laptop với bộ dữ liệu đọc từ đĩa không phải thứ chạy trên robot. Ba khác biệt.

Một — thời gian thật. Camera gửi khung ở 30 Hz bất kể bạn đã xử lý xong hay chưa. Không có nút tạm dừng.

Hai — tài nguyên hữu hạn. Một SoC nhúng có vài watt, vài gigabyte RAM, và không có quạt.

Ba — và đây là khác biệt ít được nói nhất: hệ thống là một phần của một hệ thống lớn hơn. Bộ điều khiển cần tư thế đúng lúc; nó không chờ được. Một tư thế chính xác đến sau khi cần thì vô dụng.

Ai gặp bài toán này? Ai đưa SLAM lên thiết bị. Và đây là phần mà kiến thức thuộc về kỹ thuật phần mềm hệ thống chứ không thuộc về thị giác máy tính — một lý do nữa vì sao nó thiếu trong tài liệu SLAM.

Có một cách nhìn xuyên suốt chương, và nó gọn: *mọi thứ trong chương này là về việc làm cho trường hợp xấu nhất có giới hạn.* Không phải làm cho trung bình nhanh hơn — trung bình gần như luôn đủ nhanh. Cái gây ra thất bại là một khung mất một giây, một lần cấp phát gây phân mảnh, một lần throttling làm mọi thứ chậm gấp đôi.

TRỰC GIÁC

Hình dung bạn đi xe đạp và cần nhìn đường.

Nếu bạn nhắm mắt trung bình 0,1 giây mỗi lần chớp mắt, không sao — bạn vẫn đạp được cả ngày.

Nếu thỉnh thoảng bạn nhắm mắt *hai giây*, bạn sẽ đâm vào cột điện. Và điều quan trọng: việc trung bình chỉ có 0,1 giây *không cứu bạn* ở lần hai giây ấy. Các lần nhanh không bù được cho lần chậm.

Đó là toàn bộ lý do phân vị quan trọng hơn trung bình. Với một hệ chỉ cần cho ra kết quả cuối cùng — một phép quét để in 3D chẳng hạn — trung bình là đại lượng đúng, vì tổng thời gian là thứ ta quan tâm. Với một hệ nằm trong vòng điều khiển, mỗi chu kỳ là một hạn chót riêng, và trễ một lần là hỏng một lần.

Có một chuyện thứ hai, cũng về sự bất đối xứng. Nếu bạn phải chọn giữa “nhìn kỹ và biết chính xác mình đang ở đâu, nhưng chậm” và “nhìn lướt và biết đại khái, nhưng kịp”, thì với xe đạp, lựa chọn thứ hai đúng. Một câu trả lời gần đúng đúng lúc hữu ích hơn một câu trả lời chính xác đến muộn. Đây là nguyên tắc chi phối cách thiết kế kiến trúc ở mục 3.

Và chuyện thứ ba, ít ai nghĩ tới. Đạp xe lâu thì bạn nóng người và mệt, và khi mệt thì bạn nhìn kém đi. Cái máy cũng vậy: chạy lâu thì nóng, nóng thì tự giảm tốc, và — điều bất ngờ

— nóng còn làm *ống kính giãn ra*, tức làm sai chính phép đo. Một vòng lặp vật lý mà không tài liệu phần mềm nào nhắc tới.

2.2 Thời gian: phân vị, không phải trung bình

A. Vì sao trung bình không đủ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất: đại lượng chưa đo là *phân vị 99 của thời gian xử lý*.

Lập luận. Một hệ trong vòng điều khiển có một hạn chót *mỗi chu kỳ*. Nếu chu kỳ là 33 ms và một khung mất 200 ms, thì sáu chu kỳ điều khiển không có dữ liệu mới. Bộ điều khiển phải ngoại suy, và với robot đang di chuyển, sáu chu kỳ ngoại suy có thể là mười centimet sai lệch.

Và điều quan trọng: *các khung nhanh không bù được*. Trung bình là một đại lượng cộng gộp, phù hợp khi ta quan tâm tổng; ở đây mỗi chu kỳ là một sự kiện độc lập với một hạn chót riêng.

Ba nguồn của các khung chậm, theo thứ tự phổ biến trong SLAM.

Một — *keyframe*. Khi một khung được chọn làm keyframe, local mapping chạy, và chi phí nhảy vọt. Nếu nó chạy trên cùng luồng với tracking thì mỗi keyframe là một lần trễ.

Hai — *loop closure*. Hiếm hơn nhưng đắt hơn nhiều: pose graph cộng cập nhật bản đồ có thể mất hàng giây.

Ba — *cấp phát bộ nhớ và thu gom*. Một lần cấp phát lớn có thể mất mili giây và có thời gian không xác định.

B. Vì sao tối ưu trung bình có thể làm tệ đi

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một cái bẫy thật.

Ví dụ cụ thể: bạn thêm một bộ nhớ đệm để tránh tính lại một thứ. Trung bình giảm vì hầu hết khung trúng đệm. Nhưng khi trượt đệm, chi phí cao hơn trước — vì giờ phải tính lại *và* cập nhật đệm. Phân vị 99 tăng lên.

Ví dụ thứ hai, phổ biến hơn: bạn tăng số đặc trưng để chính xác hơn. Trung bình tăng một chút. Nhưng trong các khung khó — nhiều đặc trưng hơn nghĩa là nhiều ứng viên hơn nghĩa là RANSAC lặp lâu hơn — phân vị 99 tăng nhiều.

Quy tắc thực hành: *đo phân vị 99 trước và sau mọi thay đổi hiệu năng*, và coi một thay đổi làm tăng phân vị 99 là một hồi quy kể cả khi trung bình giảm.

C. Xử lý mất khung và độ trễ

Nguyên tắc quan trọng nhất: *không bao giờ được chặn*. Nếu tracking chưa xử lý xong khung này và khung tiếp theo tới, hành vi đúng là *bỏ khung* chứ không phải xếp hàng.

Vì sao xếp hàng sai: hàng đợi làm độ trễ tăng dần và không bao giờ phục hồi. Sau một khung chậm, mọi khung sau đều trễ một chút, và độ trễ tích lũy. Với một vòng điều khiển, một tư thế đúng nhưng cũ 300 ms tệ hơn một tư thế kém chính xác hơn nhưng mới.

Ba chi tiết cài đặt đáng nhớ. *Một*: hàng đợi giữa các luồng nên có kích thước *một*, và ghi đè thay vì xếp hàng. *Hai*: mỗi khung phải mang *mốc thời gian thu*, không phải thời gian xử lý — mọi nhân tử trong đồ thị dùng mốc thu. *Ba*: đầu ra phải kèm *tuổi* của nó, để module phía trên biết mà chiết khấu; đây là phần mở rộng của nguyên tắc ở D/19 mục 6C.

2.3 Kiến trúc đa luồng và tính tất định

A. Ba luồng, ba nhịp độ

C/13 mục 4A nêu ba nhịp độ: tracking ở tần số khung, local mapping ở tần số keyframe, loop closing hiếm và lâu. Chương này bàn chi tiết kỹ thuật.

Nguyên tắc phân chia: *luồng có hạn chót cứng phải được cô lập*. Tracking có hạn chót; hai luồng kia không. Nên tracking không bao giờ được chờ hai luồng kia, kể cả khi nó cần dữ liệu mà chúng đang sửa.

Ba mẫu thiết kế giải quyết điều đó.

Bản sao đọc. Tracking làm việc trên một bản sao của bản đồ cục bộ, được cập nhật ở các điểm an toàn. Tồn bộ nhớ, nhưng tracking không bao giờ bị chặn.

Hoán đổi con trỏ. Local mapping xây dựng phiên bản mới của một cấu trúc rồi hoán đổi con trỏ trong một thao tác nguyên tử. Tracking thấy hoặc phiên bản cũ hoặc mới, không bao giờ thấy trạng thái nửa vời.

Hàng đợi một chiều. Tracking gửi keyframe cho local mapping qua một hàng đợi và *không chờ kết quả*. Kết quả quay lại qua một trong hai cơ chế trên.

B. Tính tất định là một yêu cầu, không phải xa xỉ

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và là luận điểm mà tôi cho là quan trọng nhất của chương.

Kịch bản: một robot ở hiện trường gặp lỗi. Bạn có log đầy đủ — mọi ảnh, mọi phép đo IMU, mọi mốc thời gian. Bạn chạy lại và *không tái hiện được*.

Nguyên nhân có khả năng nhất: *hệ thống không tất định*. Thứ tự luồng khác nhau làm keyframe khác được chọn; RANSAC với seed khác cho inlier khác; một khung bị bỏ ở lần chạy này mà không bị bỏ ở lần kia.

Hậu quả: bạn không debug được. Bạn có thể đoán, sửa, và không bao giờ biết mình đã sửa đúng chưa. Với một lỗi xảy ra một lần trong nghìn giờ, đây là sự khác biệt giữa sửa được và không.

Ba việc phải làm, và từ đầu dự án chứ không phải khi gặp lỗi.

Một — seed cố định và có thể tái lập. Mọi nguồn ngẫu nhiên (RANSAC, khởi tạo) phải lấy seed từ một nguồn ghi vào log.

Hai — chế độ chạy lại tất định. Một chế độ trong đó dữ liệu được đưa vào theo mốc thời gian ghi trong log, không theo thời gian thực, và các luồng chạy theo một thứ tự cố định. Chế độ này chạy chậm hơn thời gian thực và điều đó chấp nhận được — mục đích của nó là chẩn đoán.

Ba — ghi lại mọi quyết định không tất định. Khung nào bị bỏ, keyframe nào được chọn, cổng nào đóng mở. Nếu không tái lập được hoàn toàn, ít nhất có thể tái lập các quyết định.

Cái giá của tính tất định là thật: nó hạn chế một số tối ưu (xử lý song song không thứ tự), và nó có thể chậm hơn vài phần trăm. Đó là một cái giá đáng trả, và nó là một trong những lý do chính đáng nhất để chấp nhận chạy chậm hơn một chút — cùng lập luận với việc chọn ORB thay vì một front-end học được ở B/07 mục 2B.

2.4 Ngân sách tính toán và bộ nhớ

A. Biết chi phí nằm ở đâu

Trước khi tối ưu, đo. Hồ sơ chi phí điển hình của một hệ SLAM thị giác, theo thứ tự thường gặp: trích và mô tả đặc trưng thường là mục lớn nhất; khớp và RANSAC; bundle adjustment cục bộ; và các phần còn lại.

Tôi *không* đưa ra tỉ lệ phần trăm cụ thể, vì chúng phụ thuộc hoàn toàn vào độ phân giải, số đặc trưng, kích thước cửa sổ và phần cứng. Điều đáng nói là một nguyên tắc: *đo trước khi tối ưu*, và đo trên *phần cứng mục tiêu* — hồ sơ chi phí trên laptop x86 và trên SoC ARM khác nhau đáng kể, vì tỉ lệ giữa tốc độ CPU và băng thông bộ nhớ khác nhau.

B. Các kỹ thuật, theo thứ tự tỉ lệ lợi ích trên công sức

Giảm công việc trước khi tăng tốc công việc. Cách rẻ nhất để làm một phép tính nhanh hơn là không làm nó. Ví dụ cụ thể trong SLAM: thu hẹp vùng tìm kiếm bằng tiên nghiệm chuyển động (B/07 mục 5C) giảm chi phí khớp hàng chục lần; chọn keyframe tiết kiệm (C/13 mục 2) giảm chi phí BA theo lũy thừa ba.

SIMD cho các vòng lặp nóng. Tính descriptor, tính khoảng cách Hamming, warp ảnh — các thao tác này song song hoá hoàn hảo ở mức dữ liệu. Lợi ích lớn và công sức vừa phải.

Vòng lặp nóng không cấp phát. Cấp phát trước mọi bộ đệm và tái sử dụng. Lợi ích chính không phải tốc độ trung bình mà là *giảm jitter* — tức đúng thứ mục 2 nói là quan trọng.

Bộ giải thừa tối ưu. Khai thác cấu trúc khối cụ thể của bài toán thay vì dùng bộ giải thừa tổng quát (A/04 mục 4). Công sức lớn, lợi ích lớn.

Độ chính xác đơn. Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Nó nhanh gấp đôi và tốn nửa bộ nhớ, nhưng nó nguy hiểm với bài toán có số điều kiện lớn — và BA có số điều kiện lớn (B/09 mục 5A). Điều kiện để làm an toàn: *dùng dạng căn bậc hai*, vốn có số điều kiện là căn bậc hai của dạng hệ chuẩn [6]. B/09 mục 5A là chương cung cấp điều kiện ấy, và đây là một ví dụ đẹp về việc một lựa chọn toán học mở ra một tối ưu hệ thống.

C. Đặt cái gì ở đâu

Trên SoC hiện đại có nhiều khối tính toán, và việc phân bổ là một quyết định thiết kế.

CPU cho logic điều khiển và bộ giải thừa — các phép toán có nhiều nhánh và truy cập bộ nhớ không đều, nơi GPU không giúp. GPU cho các thao tác dày và đều — trích đặc trưng, mạng nơ-ron, stereo dày. NPU cho mạng, khi có. ISP cho tiền xử lý — nhưng với một cảnh báo quan trọng.

Cảnh báo về ISP: nó thường thực hiện các thao tác mà SLAM không muốn — auto-exposure thay đổi giữa các khung (vấn đề cho direct, A/02 mục 5B), khử nhiễu làm mất chi tiết nhỏ (vấn đề cho detector), và gamma phi tuyến (vấn đề cho mô hình quang trắc). Và các thao tác ấy thường không tắt được hoặc không điều khiển được đầy đủ.

Quy tắc: *lấy ảnh càng gần raw càng tốt*, và nếu phải dùng ISP thì *ghi lại tham số của nó cho mỗi khung* — thời gian phơi sáng và gain phải là một phần của dữ liệu, vì A/02 cần chúng. Đây là một yêu cầu với nhà cung cấp module camera, và nó nên được nêu *trước khi chọn module*.

2.5 Điện năng, nhiệt, và một vòng lặp bị bỏ quên

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là mục mà tôi cho là đóng góp riêng của chương.

Hai cơ chế hoàn toàn khác nhau giải thích hiện tượng “chạy tốt mười phút rồi kém dần, hồi phục sau khi nghỉ”.

Cơ chế một — thermal throttling. SoC nóng lên, tần số giảm, thời gian xử lý tăng. Hệ quả theo mục 2: nhiều khung bị bỏ hơn, cửa sổ BA phải thu nhỏ, độ trễ tăng. Đây là cơ chế mà mọi kỹ sư nhúng đều biết.

Cơ chế hai — trôi hiệu chuẩn do nhiệt. Ống kính và khung giãn nở, tiêu cự đổi vài phần vạn, và với stereo thì đường cơ sở cùng góc giữa hai camera đổi. Theo A/02 mục 4C, sai số hiệu chuẩn chảy vào tư thế và cấu trúc như một sai lệch có hệ thống.

Hai cơ chế cho cùng triệu chứng và cần hai cách chữa hoàn toàn khác nhau. Cơ chế một chữa bằng quản lý nhiệt và giảm tải; cơ chế hai chữa bằng hiệu chuẩn trực tuyến (C/14 mục 3) hoặc bằng mô hình bù theo nhiệt độ.

Phép thử phân biệt, và nó rẻ: *ghi nhiệt độ vỏ máy cùng với các đại lượng giám sát.* Nếu thời gian xử lý tăng theo nhiệt thì là cơ chế một; nếu *phần dư có cấu trúc* hoặc *tham số hiệu chuẩn trực tuyến* tương quan với nhiệt thì là cơ chế hai. Hai dấu hiệu ấy độc lập, nên phép thử phân biệt được rõ ràng — và cả hai có thể xảy ra cùng lúc.

Điều đáng nói: vòng lặp “điện năng → nhiệt → trôi ống kính → trôi hiệu chuẩn → sai số thị giác” gần như không xuất hiện trong tài liệu SLAM, vì nó nằm ở giao của ba lĩnh vực — nhiệt học, quang cơ, và thị giác — và không ai sở hữu nó. Tôi nêu nó ở đây vì tôi cho rằng nó là một trong những nguyên nhân hay bị chần đoán nhầm nhất trong triển khai thật (*đây là suy luận của tôi từ A/02 mục 4C và C/14 mục 4B; tôi không có số liệu đo và không dẫn được nguồn nào đã đo vòng lặp này một cách hệ thống*).

Hệ quả thiết kế, và nó thuộc về D/21: *đặt một cảm biến nhiệt gần module camera và ghi giá trị của nó cùng mọi dữ liệu.* Chi phí vài xu; giá trị chẩn đoán rất cao; và không thêm được sau khi đã mở khuôn.

CẦN NHỚ

Bốn quy tắc của chương này, xếp theo mức quan trọng.

Một — đo phân vị, không đo trung bình. Một hệ nhanh trung bình nhưng thỉnh thoảng chậm là hệ không dùng được trong vòng điều khiển, vì các lần nhanh không bù được cho lần chậm. Coi một thay đổi làm tăng phân vị 99 là hồi quy kể cả khi trung bình giảm.

Hai — không bao giờ chặn; bỏ khung thay vì xếp hàng. Hàng đợi làm độ trễ tăng dần và không phục hồi. Một tư thế kém chính xác đúng lúc hữu ích hơn một tư thế chính xác đến muộn.

Ba — tắt định từ đầu dự án. Không có nó thì không tái hiện được lỗi hiện trường, và với một lỗi xảy ra một lần trong nghìn giờ, đó là khác biệt giữa sửa được và không. Cái giá vài phần trăm tốc độ là đáng trả.

Bốn — ghi nhiệt độ. Vài xu cho một cảm biến, và nó là thứ phân biệt được hai cơ chế suy giảm hoàn toàn khác nhau mà nếu không có thì bạn sẽ chẩn đoán nhầm.

Và một quy tắc bao trùm: **mọi thứ trong chương này là về giới hạn trường hợp xấu nhất, không về cải thiện trung bình.** Trung bình gần như luôn đủ nhanh.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Basalt [7]	Dạng căn bậc hai; chất lượng kỹ thuật phần mềm rất cao	Mục 4B — ví dụ về việc lựa chọn toán học mở ra tối ưu hệ thống

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [3]	Ba luồng với hàng đợi và khoá	Mục 3A; đọc mã để thấy các mẫu thiết kế thật, gồm cả chỗ chúng phức tạp hơn mô tả trong bài báo
VINS-Mono [8]	Tách luồng estimator và luồng loop closure	Kiến trúc gọn, dễ đọc để học nguyên tắc cô lập luồng có hạn chót
Ceres [9]	Bộ giải thưa với nhiều lựa chọn ordering và linear solver	Mục 4B; thử các lựa chọn trên bài toán của bạn thay vì dùng mặc định
OpenCV	Nhiều hàm đã có phiên bản SIMD	Mục 4B; kiểm xem build của bạn có bật đúng cờ kiến trúc không — một lỗi phổ biến

MINI PROJECT SLAM · Đo hồ sơ thời gian trên phần cứng mục tiêu, rồi tấn công phân vị 99 · 4 ngày

Mục tiêu	chuyển từ “hệ của tôi chạy 30 Hz” sang “phân vị 99 của tôi là X ms, và nguồn của nó là Y ”.
Dữ liệu	một chuỗi dài (ít nhất mười phút) chạy trên <i>phần cứng mục tiêu</i> , không phải laptop. Nếu chưa có phần cứng mục tiêu, dùng thứ gần nhất và ghi rõ rằng kết quả sẽ khác.
Các bước	đo thời gian của từng giai đoạn pipeline cho <i>mỗi khung</i> và lưu lại toàn bộ; vẽ histogram và phân vị 50, 95, 99, 99,9; xác định các khung ở đuôi và <i>tìm ra chúng có gì chung</i> — có phải keyframe không, có loop closure không, có phải lần cấp phát lớn không; ghi nhiệt độ trong suốt và vẽ thời gian xử lý theo nhiệt độ; rồi chọn <i>một</i> nguồn ở đuôi và sửa nó, đo lại.
Xong khi	bạn có một biểu đồ phân vị trước và sau, và một lời giải thích cho <i>từng</i> đỉnh ở đuôi. Điều quan trọng nhất: bạn phải trả lời được câu “khung chậm nhất của tôi chậm vì cái gì” cho ít nhất ba nguồn khác nhau. Nếu đường cong thời gian theo nhiệt độ có xu hướng tăng, bạn vừa đo được cơ chế một ở mục 5 trên chính thiết bị của mình.
Nối với	D/21-tu-prototype-len-mass-production biến các đại lượng này thành giám sát fleet; D/19-ben-vung-trong-the-gioi-thuc mục 6 dùng chúng làm đầu vào cho phát hiện thất bại.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Đại lượng chưa đo là *phân vị 99 của thời gian xử lý*. Một hệ trong vòng điều khiển có hạn chót *mỗi chu kỳ*, và các khung nhanh không bù được cho khung chậm — trung bình là đại lượng cộng gộp, phù hợp khi ta quan tâm tổng, nhưng ở đây mỗi chu kỳ là một sự kiện độc lập với hạn chót riêng. Nếu một khung mất 200 ms thì sáu chu kỳ điều khiển không có dữ liệu mới, và với robot đang di chuyển đó có thể là mười centimet sai lệch. Ba nguồn thường gặp của khung chậm: keyframe kích hoạt local mapping, loop closure, và cấp phát bộ nhớ.
- A2. Ví dụ: bạn thêm một bộ nhớ đệm. Trung bình giảm vì hầu hết khung trúng đệm, nhưng khi *trượt* đệm thì chi phí cao hơn trước — phải tính lại *và* cấp nhật đệm — nên phân vị 99 tăng. Ví dụ thứ hai phổ biến hơn: tăng số đặc trưng để chính xác hơn làm trung bình tăng một chút, nhưng trong các khung khó thì nhiều ứng viên hơn nghĩa là RANSAC lặp lâu hơn, và phân vị 99 tăng nhiều. Quy tắc: đo phân vị 99 trước và sau mọi thay đổi hiệu năng, và coi việc nó tăng là một hồi quy kể cả khi trung bình giảm.
- A3. Điều kiện: *dùng dạng căn bậc hai* thay vì hệ chuẩn. Vì $\text{cond}(\Lambda) = \text{cond}(R)^2$, việc dựng Λ tương minh phá huỷ một nửa số chữ số có nghĩa — với bài toán BA có số điều kiện 10^8 , độ chính xác đơn (khoảng bảy chữ số) không đủ. Làm việc với R giữ lại số điều kiện ở mức căn bậc hai, và [6] cho thấy điều đó cho phép dùng độ chính xác đơn trong nhiều trường hợp. Chương cung cấp điều kiện là B/09 mục 5A — một ví dụ đẹp về việc một lựa chọn toán học mở ra một tối ưu hệ thống.

- A4. Nguyên nhân có khả năng nhất: *hệ thống không tắt định* — thứ tự luồng khác làm keyframe khác được chọn, RANSAC với seed khác cho inlier khác, một khung bị bỏ ở lần này mà không bị ở lần kia. Hậu quả: bạn không debug được; bạn đoán, sửa, và không bao giờ biết đã sửa đúng chưa. Lẽ ra phải ngăn từ đầu dự án, bằng ba việc: seed cố định ghi vào log; một chế độ chạy lại tắt định trong đó dữ liệu vào theo mốc thời gian ghi trong log và luồng chạy theo thứ tự cố định; và ghi lại mọi quyết định không tắt định (khung nào bị bỏ, keyframe nào được chọn). Cái giá vài phần trăm tốc độ là đáng trả.
- A5. Hai cơ chế. (1) *Thermal throttling*: SoC nóng, tần số giảm, thời gian xử lý tăng, nhiều khung bị bỏ hơn. (2) *Trôi hiệu chuẩn do nhiệt*: ống kính và khung giãn nở, tiêu cự đổi, và với stereo thì đường cơ sở cùng góc giữa hai camera đổi — sai số hiệu chuẩn chảy vào tư thế như một sai lệch có hệ thống (A/02 mục 4C). Hai cơ chế cho cùng triệu chứng và cần hai cách chữa ngược nhau: quản lý nhiệt so với hiệu chuẩn trực tuyến. Phép thử phân biệt rõ: ghi nhiệt độ vỏ máy; nếu *thời gian xử lý tăng* theo nhiệt thì là (1), nếu *phần dư có cấu trúc* hoặc tham số hiệu chuẩn trực tuyến tương quan với nhiệt thì là (2). Cả hai có thể xảy ra cùng lúc.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về việc đạp xe và cần nhìn đường.

Nếu trung bình bạn chớp mắt một phần mười giây, không sao cả. Nhưng nếu thỉnh thoảng bạn nhắm mắt hai giây, bạn sẽ đâm vào cột điện — và việc trung bình chỉ có một phần mười giây *không cứu bạn* ở lần hai giây ấy.

Đó là chuyện quan trọng nhất của chương. Khi một cái máy phải trả lời liên tục cho một thứ khác đang chuyển động, thì thứ đáng lo không phải là nó trả lời nhanh bao nhiêu *trung bình*, mà là lần chậm nhất của nó chậm bao lâu. Các lần nhanh không bù được cho lần chậm.

Chuyện thứ hai cũng về sự lựa chọn giữa nhanh và đúng. Nếu phải chọn giữa “nhìn thật kỹ rồi mới trả lời” và “nhìn lướt nhưng trả lời kịp”, thì với người đang đạp xe, lựa chọn thứ hai đúng. Một câu trả lời gần đúng đúng lúc hữu ích hơn một câu trả lời chính xác đến muộn. Và có một hệ quả cụ thể: khi máy xử lý không kịp, đừng để nó xếp hàng — hãy để nó *bỏ qua* và nhìn tấm ảnh mới nhất. Xếp hàng thì nó sẽ luôn trả lời về chuyện đã xảy ra vài giây trước.

Chuyện thứ ba ít ai nghĩ tới nhưng lại là chuyện tốn nhiều thời gian nhất. Giả sử máy hỏng ở nhà khách hàng và bạn có đầy đủ ghi chép về mọi thứ nó thấy. Bạn chạy lại y hệt trên máy của mình — và nó chạy bình thường. Chuyện này xảy ra khi máy *không chạy giống nhau qua hai lần*, vì có chỗ nào đó dựa vào may rủi hoặc vào thứ tự các việc chạy song song. Lúc đó bạn không thể sửa, vì bạn không tái hiện được lỗi. Và cách duy nhất tránh là làm cho nó chạy giống hệt nhau *ngay từ đầu* — thêm vào sau thì gần như không làm được.

Và chuyện cuối, thuộc về vật lý chứ không phải phần mềm. Chạy lâu thì máy nóng. Nóng thì nó tự chạy chậm lại — điều ai cũng biết. Nhưng nóng còn làm *ống kính giãn ra một chút*, tức làm sai chính phép đo — điều gần như không ai nhắc. Hai chuyện này gây ra cùng một triệu chứng: máy chạy tốt mười phút rồi kém dần, rồi hồi phục sau khi nghỉ. Chúng cần hai cách chữa hoàn toàn khác nhau, và cách phân biệt chỉ tốn một cái cảm biến nhiệt giá vài xu.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *phải quyết định các chuyện này từ rất sớm*. Tính tắt định không thêm vào được sau; cảm biến nhiệt không gắn thêm được sau khi đã mở khuôn nhựa. Đó là những quyết định phải đưa ra lúc chưa ai thấy chúng cần thiết, và đó là lý do chúng hay bị bỏ qua.

Một câu: *Đừng hỏi trung bình bao nhanh — hỏi lần chậm nhất chậm bao lâu.*

2.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *phân vị quan trọng hơn trung bình* — đúng cho hệ nằm trong vòng điều khiển và *sai* cho hệ xử lý ngoại tuyến, nơi tổng thời gian là thứ đáng quan tâm. Nó cũng yếu đi nếu module phía trên có khả năng ngoại suy tốt và chịu được độ trễ — một bộ điều khiển có mô hình

động học tốt có thể chịu được vài chu kỳ thiếu dữ liệu. Nên phát biểu chặt hơn: *phân vị quan trọng theo tỉ lệ với mức mà module phía trên không chịu được độ trễ*.

Mệnh đề thứ hai — *tính tất định là yêu cầu bắt buộc* — bị bác bỏ nếu trong thực tế, phần lớn lỗi hiện trường tái hiện được mà không cần nó, hoặc nếu các kỹ thuật ghi log đủ chi tiết thay thế được. Tôi cho là không, nhưng mức độ phụ thuộc vào tần suất các lỗi hiếm — và với một hệ ổn định thì có thể chúng đủ hiếm để không đáng đầu tư. Đây là một đánh đổi phụ thuộc dự án.

Mệnh đề thứ ba — *vòng lặp nhiệt gây trôi hiệu chuẩn* — là suy luận của tôi từ hai chương khác, và tôi đã ghi rõ rằng *không có số liệu đo và không dẫn được nguồn nào đã đo nó một cách hệ thống*. Đây là mệnh đề có rủi ro sai cao nhất trong chương. Nó dễ kiểm: ghi nhiệt độ và tham số hiệu chuẩn trực tuyến trên một thiết bị thật trong vài giờ, và xem chúng có tương quan không. Nếu không, mệnh đề sai và tôi nên rút nó. Phép đo này nằm trong 99-chua-biet.md như một mục ưu tiên cao, vì nó rẻ và vì kết quả đổi cách chẩn đoán.

2.7 Kết nối

Nối lên trên. A/04-uoc-luong-map-va-bundle-adjustment là nguồn của cả chi phí lẫn cơ hội tối ưu lớn nhất — cấu trúc thừa. B/09-paradigm-uoc-luong-back-end mục 5A cung cấp điều kiện cho việc dùng độ chính xác đơn, và nói chung là chỗ lựa chọn kiến trúc quyết định hồ sơ chi phí. C/13-quan-ly-ban-do mục 4A nêu kiến trúc ba luồng mà chương này bàn chi tiết kỹ thuật.

Nối ngang. D/19-ben-vung-trong-the-gioi-thuc mục 6 đề nghị bốn tầng giám sát; chương này nhắc rằng chúng không miễn phí và phải nằm trong ngân sách. Ngược lại, mục 5 ở đây cho D/19 một chế độ hồng thuộc nhóm bốn mà chương đó không nêu: trôi do nhiệt theo chu kỳ hoạt động chứ không theo tuổi thiết bị.

Nối xuống dưới. D/21-tu-prototype-len-mass-production là chỗ các quyết định ở đây trở thành yêu cầu phân cứng: chọn SoC theo ngân sách nhiệt chứ không chỉ theo hiệu năng đỉnh, yêu cầu truy cập ảnh gần raw từ nhà cung cấp module, và đặt cảm biến nhiệt gần camera. Cả ba phải quyết *trước khi chốt thiết kế*, và đó là lý do chương 21 khuyến nghị đọc chương này sớm hơn thứ tự tự nhiên của nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao trung bình không đủ cho một hệ trong vòng điều khiển, và nêu ba nguồn thường gặp của các khung ở đuôi.
2. Nêu một thay đổi làm giảm thời gian trung bình mà tăng phân vị 99, và giải thích cơ chế.
3. Vì sao bỏ khung đúng hơn xếp hàng? Nêu ba chi tiết cài đặt đi kèm nguyên tắc đó.
4. Nêu ba việc phải làm để có tính tất định, và nói vì sao chúng phải làm từ đầu dự án.
5. Điều kiện để dùng độ chính xác đơn an toàn là gì, và chương nào cung cấp điều kiện ấy?
6. Nêu cảnh báo về ISP và hai yêu cầu nên đặt ra với nhà cung cấp module camera.
7. Hai cơ chế gây “chạy tốt rồi kém dần rồi hồi phục” là gì, và phép thử nào phân biệt chúng?

Nguồn

- [1] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [2] J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [6] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [7] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [8] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [9] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.
- [10] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [13] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.
- [14] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.

- [15] J. Maye, P. Furgale, and R. Siegwart, "Self-Supervised Calibration for Robotic Systems," in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [16] T. Schneider, M. Li, C. Cadena, et al., "Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation," in *IEEE Sensors Journal*, mức tin B, 2019.

Từ prototype lên mass production (From Prototype to Mass Production)

PHỤ THUỘC C/14-hieu-chuan-nhu-mot-nhanh-cua-slam (hiệu chuẩn — ở đây nó trở thành một quy trình có chỉ tiêu thời gian và chất lượng); C/18-danh-gia-va-phuong-phap-luan (đánh giá — ở đây nó trở thành regression test và chỉ tiêu nghiệm thu); D/19-ben-vung-trong-the-gioi-thuc (chế độ hồng — ở đây thành fleet analytics); D/20-toi-uu-hieu-nang-va-he-thong (ràng buộc phần cứng — nên đọc trước chương này).

TÓM TẮT

Đây là chương duy nhất trong cây viết cho mục tiêu sản xuất hàng loạt, và nó là chương ít tài liệu nhất. Nội dung tổ chức theo bốn nhóm, tương ứng bốn nhóm quyết định mà một dự án phải đưa ra. A — *hiệu chuẩn ở quy mô sản xuất*: đồ gá tự động, chu trình dưới một phút, dung sai cơ khí, trôi theo thời gian, và một quyết định tốn nhiều tiền là hiệu chuẩn từng máy hay dùng tham số chung cho cả lô. B — *độ tin cậy*: định nghĩa chế độ hồng, giám sát sức khỏe tại hiện trường, watchdog, logging, và fleet analytics — vòng lặp biến hàng nghìn máy thành một công cụ nghiên cứu. C — *kiểm thử*: bộ dữ liệu nội bộ, regression tự động, hardware-in-the-loop, ma trận thử nghiệm, và nguyên tắc không bao giờ tin benchmark công khai là đủ. D — *ràng buộc sản phẩm*: chọn cấu hình cảm biến, chi phí BOM, nhiệt, điện, OTA, quyền riêng tư. Điều quan trọng nhất về chương này là *thứ tự*: phần lớn các quyết định ở nhóm A và D phải đưa ra *trước khi chốt thiết kế cơ khí*, khi chưa ai thấy chúng cần thiết — và không sửa được sau khi mở khuôn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đồ gá hiệu chuẩn (calibration fixture)** — thiết bị cơ khí giữ sản phẩm và target ở các tư thế đã biết, để quy trình hiệu chuẩn chạy tự động và lặp lại được trên dây chuyền.
- **golden sample** — một hoặc vài đơn vị sản phẩm được đo rất kỹ và giữ làm chuẩn tham chiếu, dùng để phát hiện trôi của chính quy trình sản xuất và quy trình đo.
- **SPC** — *statistical process control* — theo dõi thống kê của một tham số qua các lô sản xuất, để phát hiện khi quy trình bắt đầu trôi *trước khi* sản phẩm bị lỗi.
- **phân tích độ nhạy** — đo xem sai lệch của một tham số đầu vào (dung sai cơ khí, sai số hiệu chuẩn) gây ra sai lệch bao nhiêu ở đầu ra, để biết dung sai nào cần siết và dung sai nào nới được.
- **fleet analytics** — thu thập và phân tích dữ liệu vận hành từ toàn bộ đội thiết bị đang hoạt động, để tìm ra các chế độ hồng mà phòng thí nghiệm không tái hiện được.
- **HIL** — *hardware-in-the-loop* — chạy phần mềm trên phần cứng thật với dữ liệu được phát lại, để kiểm thử mà không cần môi trường thật.
- **MTBF** — *mean time between failures* — thời gian trung bình giữa hai lần hỏng. Với SLAM, cần định nghĩa rõ “hỏng” nghĩa là gì trước khi con số này có nghĩa.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn có một nguyên mẫu chạy tốt và cần sản xuất một nghìn máy. Nếu quyết định quan trọng nhất phải đưa ra, và nói vì sao nó phải đưa ra *trước* khi chốt thiết kế cơ khí.

- Q2.** Hiệu chuẩn từng máy mất ba phút và tốn tiền; dùng tham số chung cho cả lô thì miễn phí. Nếu đại lượng quyết định lựa chọn ấy, và cách đo nó.
- Q3.** Hệ thống của bạn chạy hoàn hảo trên EuRoC và trên bộ dữ liệu nội bộ. Nếu ba lý do vì sao nó vẫn có thể thất bại ở nhà khách hàng đầu tiên.
- Q4.** Một nghìn máy đang chạy và bạn nhận được một trăm báo cáo lỗi mỗi tháng, phần lớn mơ hồ. Nếu thứ bạn cần thiết kế *từ trước* để biến các báo cáo ấy thành thông tin sửa được.
- Q5.** Bạn phải chọn cấu hình cảm biến cho một robot dịch vụ trong nhà. Trình bày cách biến câu hỏi này thành một bài toán có tiêu chí, và nêu kết luận cùng điều kiện của nó.

3.1 Đặt vấn đề

Ba phần trước và hai chương trước dựng một hệ thống chạy được. Chương này về việc biến nó thành một nghìn hệ thống chạy được ở một nghìn nơi khác nhau, không có bạn ở đó.

Bài toán gốc: khoảng cách giữa *một máy chạy được* và *mọi máy chạy được* lớn hơn nhiều so với trực giác, và nó lớn theo những hướng mà kỹ thuật thuật toán không chạm tới.

Ba nguồn của khoảng cách ấy.

Một — biến thiên giữa các đơn vị. Nguyên mẫu của bạn có một camera cụ thể lắp ở một vị trí cụ thể. Một nghìn máy có một nghìn camera hơi khác nhau, lắp ở các vị trí hơi khác nhau trong dung sai cơ khí. Tham số hiệu chuẩn của máy này không dùng cho máy kia — hoặc có dùng được, và câu hỏi “có hay không” là một quyết định tốn nhiều tiền (mục 2C).

Hai — biến thiên giữa các môi trường. Bạn thử trong văn phòng của mình. Khách hàng dùng nó trong một nhà kho có sàn bóng, một căn hộ có nhiều gương, một nhà xưởng có bụi. Mỗi môi trường kích hoạt một mục khác trong danh sách ở [D/19](#).

Ba — biến thiên theo thời gian. Nguyên mẫu chạy vài giờ. Sản phẩm chạy vài năm, trong đó ống kính bám bụi, keo lão hoá, môi trường thay đổi, và phần mềm được cập nhật.

Ai gặp bài toán này? Chỉ những người thật sự sản xuất, và đó là lý do tài liệu về nó gần như không tồn tại trong học thuật. Kiến thức thuộc về các công ty; nó không tổng quát hoá sạch sẽ thành định lý; và mỗi ràng buộc là riêng của một sản phẩm.

CẠM BẮY

Toàn bộ chương này là nguyên tắc và danh mục kiểm, không phải số liệu.

Tôi *không* có dữ liệu fleet thật, và mọi con số xuất hiện trong chương — “dưới sáu mươi giây”, “một nghìn máy”, các tỉ lệ phần trăm — là **ví dụ minh họa tự đặt** để lập luận có chỗ bám, không phải kết quả đo. Theo `learning-rules.md` mục 5, tôi phải nói rõ điều này một lần ở đầu chương thay vì rải cảnh báo khắp nơi.

Điều tôi đưa ra là *cấu trúc của các quyết định*: quyết định nào phải đưa ra trước quyết định nào, đại lượng nào quyết định mỗi lựa chọn, và cách đo đại lượng ấy. Cấu trúc ấy tôi tin là đúng và nó suy ra được từ các chương trước. Các con số thì phải do chính dự án của bạn đo.

Và một cảnh báo về cách đọc: chương này liệt kê rất nhiều việc phải làm. Không dự án nào làm hết ngay từ đầu. Cách dùng đúng là coi nó như một *danh mục để quyết định bỏ qua có ý thức* — biết rằng bạn đang bỏ qua điều gì và vì sao, thay vì phát hiện nó vào lúc đã quá muộn để sửa.

TRỰC GIÁC

Hình dung sự khác nhau giữa nấu một bữa ăn ngon và mở một nhà hàng.

Nấu một bữa ngon: bạn nêm, bạn chỉnh, bạn dùng đúng nguyên liệu mình thích. Kết quả phụ thuộc vào tay nghề của bạn ở thời điểm đó.

Mở nhà hàng thì khác về bản chất. Món ăn phải giống nhau ở mọi bàn, mọi ngày, do nhiều đầu bếp khác nhau nấu, với nguyên liệu mua từ nhiều nguồn khác nhau. Bạn không thể nêm từng đĩa.

Từ đó ra ba việc mà người nấu ăn ở nhà không bao giờ nghĩ tới, và chúng ánh xạ chính xác sang ba nhóm trong chương này.

Một — công thức phải đo được, không phải “nêm vừa ăn”. Bao nhiêu gram, bao nhiêu phút. Vì người nấu không phải bạn, và “vừa ăn” của họ khác của bạn. Đây là hiệu chuẩn và quy trình.

Hai — phải biết món nào bị hỏng mà không cần nêm. Nhiệt độ bếp, màu sắc, thời gian — các dấu hiệu gián tiếp thay cho việc nêm. Đây là giám sát sức khỏe.

Ba — phải có cách biết khách hàng nghĩ gì. Không phải mỗi lời phàn nàn lẻ tẻ, mà là: món nào bị trả lại nhiều nhất, vào giờ nào, ở bàn nào. Đây là fleet analytics, và nó là thứ biến một trăm lời phàn nàn mơ hồ thành một chẩn đoán.

Và có một việc thứ tư, quan trọng hơn cả ba và dễ bỏ quên nhất: *bạn phải quyết định kích thước bếp trước khi xây nhà hàng.* Sau khi đã xây, muốn thêm một cái lò nữa thì phải đập tường. Trong sản phẩm, những quyết định kiểu “đập tường” là dung sai cơ khí, chỗ đặt cảm biến, và loại camera — và chúng phải quyết vào lúc chưa ai thấy chúng quan trọng.

3.2 A – Hiệu chuẩn ở quy mô sản xuất

A. Phân tích độ nhạy phải chạy trước

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là quyết định quan trọng nhất trong chương.

Câu hỏi: *dung sai cơ khí bao nhiêu là chấp nhận được?* Cụ thể: hai camera stereo lệch góc bao nhiêu độ thì vẫn đạt yêu cầu độ chính xác? Module camera xoay bao nhiêu so với thân máy?

Câu hỏi ấy *phải* được trả lời trước khi chốt thiết kế cơ khí, vì siết dung sai sau khi mở khuôn là bất khả về mặt kinh tế.

Quy trình trả lời, và nó là một quy trình chạy được chứ không phải một lời khuyên:

Bước một — xác định ngân sách sai số đầu ra. Từ yêu cầu sản phẩm: robot phải định vị chính xác bao nhiêu để làm được việc của nó.

Bước hai — lập danh sách các tham số đầu vào và độ tán dự kiến của chúng. Với mỗi tham số, một khoảng: tiêu cự lệch bao nhiêu giữa các đơn vị, góc lắp lệch bao nhiêu, đường cơ sở lệch bao nhiêu.

Bước ba — chạy Monte-Carlo. Nhiễu loạn từng tham số theo độ tán của nó trong mô phỏng, đo sai số đầu ra. Đây là C/18 mục 3B áp dụng cho một mục đích khác: mô phỏng để tách các nguồn.

Bước bốn — đọc kết quả theo hai chiều. Tham số nào mà sai lệch nhỏ gây sai số lớn thì *siết dung sai* hoặc *hiệu chuẩn từng máy*. Tham số nào mà sai lệch lớn cũng không sao thì *nới dung sai* để tiết kiệm chi phí cơ khí.

Chiều thứ hai là chiều hay bị bỏ quên và nó tiết kiệm tiền thật: phần lớn dự án siết mọi dung sai vì không biết cái nào quan trọng, và điều đó đắt một cách không cần thiết.

B. Quy trình hiệu chuẩn trên dây chuyền

Bốn ràng buộc mà quy trình phòng lab không có.

Thời gian. Mỗi giây trên dây chuyền là tiền. Một quy trình dùng Kalibr với người thao tác cầm bảng đi lại mất vài phút và không lặp lại được — không dùng được.

Tính lặp lại. Kết quả không được phụ thuộc vào người thao tác. Điều này đòi hỏi một *đồ gá*: một cơ cấu giữ sản phẩm và di chuyển target (hoặc ngược lại) qua một chuỗi tư thế cố định.

Chuỗi tư thế phải thoả điều kiện kích thích. Đây là chỗ A/05 Bảng kích thích và C/14 trả công trực tiếp. Chuỗi tư thế của đồ gá phải được *thiết kế* để mỗi tham số cần ước lượng đều được kích thích: đa dạng độ sâu cho tiêu cự, target ở bốn góc ảnh cho điểm chính và méo, xoay quanh nhiều trục nếu có hiệu chuẩn camera-IMU.

Điều đáng nhấn mạnh: chuỗi ấy nên được *kiểm bằng phân tích observability* trước khi chế tạo đồ gá — dựng bài toán MAP với chuỗi tư thế dự kiến và xem hiệp phương sai của các tham số. Nếu một tham số có hiệp phương sai lớn thì chuỗi chưa đủ, và phát hiện điều đó trên máy tính rẻ hơn nhiều so với phát hiện sau khi đã chế tạo đồ gá.

Tiêu chí đạt/không đạt tự động. Không có kỹ sư nhìn từng kết quả. Cần một bộ tiêu chí chạy tự động, và theo A/02 mục 4 thì RMS *không* phải tiêu chí đúng. Bốn tiêu chí đúng: độ phủ của target trên khung hình (kiểm được *trước* khi tối ưu — rẻ nhất, nên chạy đầu tiên); hiệp phương sai của từng tham số; hoa văn phần dư; và so với phân bố của lô (mục 2D).

C. Hiệu chuẩn từng máy hay dùng chung

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một quyết định có ảnh hưởng lớn tới chi phí.

Đại lượng quyết định là một *tỉ số*:

$$\frac{\text{độ tán của tham số trong lô}}{\text{mức tham số đó được phép sai theo ngân sách}}.$$

Tử số đo bằng cách hiệu chuẩn một mẫu vài chục máy và tính độ lệch chuẩn. Mẫu số đến từ phân tích độ nhảy ở mục 2A.

Nếu tỉ số nhỏ hơn một, tham số chung dùng được và bạn tiết kiệm được thời gian dây chuyền. Nếu lớn hơn một, phải hiệu chuẩn từng máy — hoặc siết dung sai cơ khí, vốn có thể rẻ hơn tùy trường hợp.

Và quyết định không nhất thiết đồng nhất cho mọi tham số. Thường gặp: *tiêu cự và méo* khá đồng đều trong một lô module cùng nguồn nên dùng chung được; *ngoại* — vị trí tương đối giữa các camera — phụ thuộc vào lắp ráp nên độ tán lớn hơn nhiều và thường phải đo từng máy. Một quy trình hỗn hợp — dùng chung nội, đo riêng ngoại — thường là điểm tối ưu, và nó nhanh hơn nhiều so với đo tất cả.

D. Theo dõi qua thời gian: golden sample và SPC

Hai cơ chế cần thiết và hay bị bỏ.

Golden sample. Giữ vài đơn vị đã được đo rất kỹ bằng một phương pháp độc lập và chính xác hơn. Chạy chúng qua dây chuyền định kỳ. Nếu kết quả trôi, thì *quy trình đo* đang trôi — không phải sản phẩm. Không có cơ chế này thì bạn không phân biệt được “lô này kém” với “máy đo bị lệch”.

SPC trên tham số hiệu chuẩn. Vẽ phân bố của từng tham số qua các lô. Một dịch chuyển của trung bình hoặc một giãn nở của độ tán là dấu hiệu quy trình sản xuất đang trôi — *trước khi* sản phẩm bị lỗi. Đây là công cụ chuẩn của sản xuất công nghiệp, và nó áp dụng trực tiếp cho tham số hiệu chuẩn mà ít ai làm.

Trôi sau khi xuất xưởng là chuyện của C/14 mục 4B và D/20 mục 5: nhiệt độ, rung, va đập, lão hoá keo. Hai cơ chế xử lý: hiệu chuẩn trực tuyến có cổng observability, và phát hiện để yêu cầu hiệu chuẩn lại.

3.3 B – Độ tin cậy và fleet analytics

A. Định nghĩa “hỏng” trước khi đo MTBF

MTBF là một con số vô nghĩa cho tới khi “hỏng” được định nghĩa. Với SLAM, cần một thang chứ không phải một ngưỡng, và nó nên khớp với thang chế độ suy giảm ở D/19 mục 6C.

Một thang hợp lý, từ nhẹ tới nặng: suy giảm có phát hiện (hệ chuyển chế độ và báo — không tính là hỏng); mất bám tạm thời có tự phục hồi; mất bám cần khởi động lại phần mềm; sai vị trí im lặng (ngghiêm trọng nhất, vì hậu quả không giới hạn); và bản đồ bị phá (mất dữ liệu).

Mỗi mức có một tỉ lệ chấp nhận khác nhau, và *sai vị trí im lặng* nên có tỉ lệ chấp nhận thấp hơn nhiều bậc so với các mức khác — đúng lập luận ở D/19 mục 6A.

B. Giám sát sức khoẻ tại hiện trường

Các đại lượng đã được liệt kê rải rác: C/18 mục 3C, D/19 khối *Cần nhớ*, D/20 mục 2. Ở đây là ba nguyên tắc để biến chúng thành một hệ thống.

Một — đo nguyên nhân, không chỉ hậu quả (D/19). Mật độ đặc trưng theo vùng ảnh nói nhiều hơn ATE.

Hai — hiệu chuẩn quan hệ giữa đại lượng thay thế và sai số thật, một lần, trong lab. Các đại lượng giám sát không đo sai số; chúng đo các điều kiện mà sai số nhỏ phụ thuộc vào. Quan hệ giữa chúng phải được thiết lập trên dữ liệu có chân trị rồi mang ra dùng. Bước này bị bỏ qua gần như mọi nơi, và không có nó thì các ngưỡng cảnh báo là số đoán.

Ba — ngân sách phải nằm trong ngân sách. Theo D/20, mọi thứ chạy mỗi khung đều tốn. Phần lớn đại lượng giám sát rẻ (vài chục con số mỗi khung, tổng hợp thành thống kê mỗi giây), nhưng vài cái không — tính phổ giá trị riêng của Λ chẳng hạn. Chạy cái đắt theo chu kỳ thưa hơn.

C. Logging: đủ để tái hiện, không nhiều hơn

Đây là một phần của câu trả lời cho câu hỏi mở đầu thứ tư.

Yêu cầu mâu thuẫn: log phải đủ để tái hiện một lỗi, nhưng không được tốn băng thông, bộ nhớ, hay quyền riêng tư.

Cách giải quyết là *ba tầng*.

Tầng một — luôn bật, rất rẻ. Các đại lượng giám sát tổng hợp theo giây, kèm nhiệt độ và trạng thái chế độ. Vài trăm byte mỗi giây. Đây là thứ cho phép fleet analytics.

Tầng hai — vòng đệm quay vòng, giữ vài chục giây gần nhất. Đủ chi tiết để tái hiện: mốc thời gian, quyết định keyframe, kết quả các cổng. Chỉ được *ghi ra* khi có sự kiện đáng chú ý — một lần mất bám, một cảnh báo. Đây là cơ chế quan trọng nhất, vì nó cho bạn dữ liệu về *chính khoảnh khắc* trước khi hỏng.

Tầng ba — ghi ảnh, chỉ khi được phép. Đắt và nhạy cảm về quyền riêng tư. Chỉ bật theo yêu cầu và có sự đồng ý.

Và theo D/20 mục 3B: log tầng hai chỉ có giá trị nếu hệ thống *tất định*. Nếu không, bạn có dữ liệu mà không tái hiện được.

D. Fleet analytics: vòng lặp quan trọng nhất

Đây là phần còn lại của câu trả lời cho câu hỏi mở đầu thứ tư, và tôi cho là đóng góp lớn nhất của chương.

Một nghìn máy chạy tám tiếng mỗi ngày là tám nghìn giờ dữ liệu mỗi ngày — nhiều hơn bất kỳ bộ dữ liệu nghiên cứu nào nhiều bậc. Nếu bạn thu được các đại lượng tổng hợp từ chúng, bạn có một công cụ mà không phòng thí nghiệm nào có.

Vòng lặp, bốn bước.

Một — thu thập các đại lượng tăng một từ toàn đội, cộng các bản ghi tăng hai khi có sự kiện.

Hai — phân cụm các sự kiện theo dấu hiệu, không theo mô tả của khách hàng. Các lần mất bảm có cùng hồ sơ đại lượng giám sát — cùng kiểu sụt đặc trưng, cùng kiểu tăng phần dư — nhiều khả năng có cùng nguyên nhân, dù khách hàng mô tả chúng bằng những từ khác nhau.

Ba — xếp hạng các cụm theo tần suất nhân mức nghiêm trọng. Đây là danh sách việc cần làm, và nó khác rất nhiều so với danh sách mà kỹ sư tự nghĩ ra.

Bốn — tái hiện cụm lớn nhất trong lab bằng dữ liệu tăng hai, sửa, và đo lại trên fleet ở phiên bản sau.

Điều quan trọng nhất về vòng lặp này: nó biến “một trăm báo cáo mơ hồ” thành “ba nguyên nhân xếp theo mức quan trọng”. Và nó chỉ khởi động được nếu ba thứ đã được thiết kế *từ trước*: các đại lượng giám sát, cơ chế log ba tầng, và tính tất định. Không thứ nào thêm vào được sau một cách rẻ.

Đây cũng là chỗ [D/19](#) mục 6A trả công: một hệ biết mình sai tạo ra dữ liệu; một hệ im lặng thì không. Toàn bộ giá trị của fleet analytics phụ thuộc vào việc hệ thống phát hiện được thất bại của chính nó.

3.4 C – Kiểm thử

A. Bộ dữ liệu nội bộ

Đây là câu trả lời cho câu hỏi mở đầu thứ ba.

Ba lý do một hệ chạy hoàn hảo trên benchmark vẫn thất bại ở nhà khách hàng đầu tiên.

Một — benchmark không chứa các chế độ hỏng ở [D/19](#). Chúng được thu trong điều kiện kiểm soát, không có gương, ít người đi lại, không có bụi ống kính.

Hai — benchmark quá ngắn. Không bộ dữ liệu chuẩn nào dài tám tiếng, nên các vấn đề ở [C/13](#) mục 4 — kích thước bản đồ, lão hoá — không xuất hiện.

Ba — benchmark đo sai thứ. Chúng đo ATE trung bình trên các chuỗi hệ thống chạy được; sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi ([C/18](#) mục 5D).

Kết luận, và nó là một nguyên tắc cứng: *không bao giờ tin benchmark công khai là đủ.* Cần một bộ dữ liệu nội bộ phản ánh môi trường khách hàng, và nó nên gồm ba phần: các chuỗi *đại diện* (môi trường điển hình, dài), các chuỗi *ngịch cảnh* (bộ ở [D/19](#) mini project), và các chuỗi *từ hiện trường* (dữ liệu tăng hai từ các lần hỏng thật — đây là phần có giá trị nhất và nó chỉ có được nhờ vòng lặp ở mục 3D).

B. Regression test tự động

Mỗi commit chạy toàn bộ bộ dữ liệu nội bộ và so với đường cơ sở. Ba chi tiết quyết định nó có dùng được hay không.

Một — so với phân vị, không chỉ trung bình (C/18, D/20). Một thay đổi làm giảm ATE trung bình mà tăng tỉ lệ thất bại là một hồi quy.

Hai — chạy nhiều seed nếu hệ không tắt định; nếu tắt định thì một lần là đủ, và đó là một lý do nữa để đầu tư vào tính tắt định.

Ba — nhiều chỉ tiêu, không một. ATE, tỉ lệ thất bại, tỉ lệ phát hiện thất bại, phân vị thời gian, bộ nhớ. Một bảng chứ không một con số.

C. HIL và ma trận thử nghiệm

HIL: chạy phần mềm trên phần cứng thật với dữ liệu phát lại. Nó bắt được các vấn đề mà chạy trên máy tính bỏ sót: thermal throttling (D/20 mục 5), giới hạn bộ nhớ, hành vi của ISP. Điều kiện để có giá trị: phát lại phải đúng thời gian thực, vì phần lớn vấn đề ở đây là vấn đề thời gian.

Ma trận thử nghiệm: liệt kê các chiều biến thiên — ánh sáng, loại sàn, tốc độ, tải, mức độ động của môi trường — và thử các tổ hợp. Không thể thử hết, nên chọn theo hai tiêu chí: các tổ hợp có khả năng xảy ra ở khách hàng, và các tổ hợp mà lý thuyết nói là khó (từ A/05 và D/19).

Điều đáng nhớ: ma trận nên được xây từ hai nguồn — kinh nghiệm về khách hàng, và lý thuyết về chỗ hệ thống suy biến. Chỉ dùng nguồn thứ nhất thì bỏ sót các trường hợp chưa gặp; chỉ dùng nguồn thứ hai thì thử nhiều thứ không bao giờ xảy ra.

I 3.5 D — Ràng buộc sản phẩm

A. Chọn cấu hình cảm biến

Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Cách biến nó thành bài toán có tiêu chí, ba bước, dùng lại các công cụ đã có.

Bước một — liệt kê gauge cần phá. Từ A/05: mono thuần thị giác có bảy bậc không quan sát được. Ứng dụng cần bao nhiêu trong số đó? Robot dịch vụ cần tỉ lệ metric (để đo khoảng cách thật) và cần hướng ổn định; nó không cần vị trí tuyệt đối toàn cục.

Bước hai — với mỗi ứng viên cảm biến, xem nó phá gauge nào và đòi điều kiện gì (C/15 Bảng cảm biến).

Bước ba — kiểm tính độc lập của chế độ hỏng (D/19). Hai cảm biến cùng chết trong bóng tối không cho khả năng phục hồi.

Áp dụng cho robot dịch vụ trong nhà, kết luận của tôi: stereo + IMU + odometry bánh xe, với global shutter.

Lập luận: stereo phá gauge tỉ lệ ngay từ khung đầu, làm khởi tạo trở nên tầm thường và xoá cả một lớp chế độ hỏng (B/10 mục 3A); IMU ghim roll với pitch, cho chuyển động tần số cao, và quan trọng nhất là nó sống khi thị giác chết; odometry bánh xe cho tỉ lệ độc lập và ràng buộc nonholonomic miễn phí, và nó sống khi cả thị giác lẫn IMU đều kém (C/15 mục 4A). Ba cảm biến có ba chế độ hỏng gần như độc lập: thị giác chết trong tối, IMU trôi khi đứng yên lâu, odometry nói dối khi trượt — và chúng kiểm chéo lẫn nhau.

Global shutter: theo A/02 mục 5A, nó xoá bỏ một lớp mô hình, một lớp tham số phải hiệu chuẩn, và một lớp chế độ hỏng chỉ xuất hiện khi rung. Chênh lệch giá vài độ la mỗi máy đổi lấy điều đó là một trong số ít quyết định trong cây mà phần cứng thắng phần mềm rõ ràng.

Điều kiện của kết luận này: nó dành cho robot mặt đất, trong nhà, hoạt động không giám sát, với ngân sách cho phép ba cảm biến. Nó không áp dụng cho drone (không có bánh xe), cho thiết bị cầm tay (không có odometry), hay cho sản phẩm cực rẻ (ngân sách chỉ đủ một camera). (Đây là

suy luận của tôi từ các chương trước, không phải kết quả của một khảo sát thị trường hay một thử nghiệm so sánh.)

B. OTA, phiên bản, và quyền riêng tư

Ba ràng buộc còn lại, mỗi cái một đoạn.

Cập nhật OTA cho phần mềm, bản đồ, và mô hình. Ba yêu cầu: định dạng bản đồ phải có phiên bản (C/13 mục 5A) — thêm sau rất đầu; phải có cơ chế quay lui khi bản cập nhật gây hồi quy; và với mô hình học được, mỗi lần đổi mô hình đòi *tái kiểm định toàn bộ* (C/17), nên chi phí cập nhật cao hơn nhiều so với đổi một ngưỡng.

Quyền riêng tư. Theo C/13 mục 5B, bản đồ gồm điểm và descriptor là dữ liệu riêng tư — tái tạo được ảnh từ nó [10]. Ba hệ quả: tải bản đồ lên đám mây là truyền dữ liệu cá nhân; lưu trên thiết bị thì nó là mục tiêu; và log tầng ba (ảnh) phải có sự đồng ý rõ ràng. Điều tối thiểu: coi bản đồ là dữ liệu cá nhân trong mọi quyết định thiết kế, và nói rõ điều đó với khách hàng.

Nhiệt và điện. Theo D/20 mục 5: chọn SoC theo ngân sách nhiệt liên tục chứ không theo hiệu năng đỉnh, và *đặt một cảm biến nhiệt gần module camera*. Chi phí vài xu, giá trị chẩn đoán cao, và không gắn thêm được sau khi mở khuôn.

CẦN NHỚ

Thứ tự quan trọng hơn nội dung. Chương này liệt kê nhiều việc; điều quyết định là việc nào phải làm *trước*.

Bốn quyết định **không sửa được sau khi chốt cơ khí**, xếp theo mức tổn kém nếu sai:

Một — *dung sai cơ khí*, quyết bởi phân tích độ nhạy ở mục 2A. Sai thì hoặc phải mở khuôn lại, hoặc phải hiệu chuẩn từng máy suốt đời sản phẩm.

Hai — *loại cảm biến và loại shutter*. Global shutter và cấu hình cảm biến quyết định cả một lớp chế độ hồng có tồn tại hay không.

Ba — *các cảm biến phụ trợ rẻ tiền*: cảm biến nhiệt gần camera, encoder bánh xe. Vài xu mỗi cái, và chúng là thứ phân biệt được các nguyên nhân suy giảm.

Bốn — *truy cập ảnh gần raw* và điều khiển ISP, vốn là một yêu cầu với nhà cung cấp module phải nêu *trước khi chọn module*.

Và ba quyết định **phần mềm** nên đưa ra từ đầu vì thêm sau rất đắt: *tính tất định* (D/20 mục 3B); *định dạng bản đồ có phiên bản*; và *cơ chế giám sát cùng log ba tầng*, vì không có chúng thì vòng lặp fleet analytics ở mục 3D không khởi động được.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [11]	Hiệu chuẩn đa cảm biến; quy trình thu dữ liệu	Điểm khởi đầu cho mục 2B — nhưng quy trình dây chuyền phải tự thiết kế, vì Kalibr không nhắm tới dây chuyền
maplab [12]	Nhiều phiên, quản lý bản đồ dài hạn	Hiếm framework nghĩ về hoạt động nhiều phiên; liên quan mục 5B
OpenLORIS [4]	Bộ dữ liệu nhiều phiên trong môi trường thật có thay đổi	Gần nhất với tinh thần mục 4A trong các bộ dữ liệu công khai
evo [13], rpg_trajectory_evaluation [14]	Công cụ đánh giá quỹ đạo	Nền cho regression test ở mục 4B; nhớ chọn đúng chế độ căn chỉnh

MINI PROJECT SLAM · Chạy phân tích độ nhạy cho sản phẩm của bạn, trước khi chốt cơ khí · 4 ngày

Mục tiêu	biến câu hỏi “dung sai bao nhiêu là đủ” từ một cuộc tranh luận thành một bảng số — và làm điều đó vào lúc còn sửa được.
Dữ liệu	không cần phần cứng: toàn bộ bài này chạy trong mô phỏng, và đó chính là điểm mạnh của nó.
Các bước	viết ra ngân sách sai số đầu ra từ yêu cầu sản phẩm; lập danh sách mười tham số đầu vào với độ tán dự kiến (nếu chưa biết, dùng dung sai mà nhà cung cấp cơ khí đề xuất làm điểm khởi đầu); chạy Monte-Carlo nhiều lần từng tham số riêng lẻ rồi nhiều lần tất cả cùng lúc; xếp hạng theo độ nhạy; với ba tham số nhạy nhất, tính tỉ số ở mục 2C để quyết định siết dung sai, hiệu chuẩn từng máy, hay cả hai; cuối cùng thiết kế chuỗi tư thế cho đồ gá và kiểm nó bằng phân tích observability theo mục 2B trước khi vẽ cơ khí.
Xong khi	bạn có một bảng mười tham số với ba cột: độ nhạy, độ tán dự kiến, và quyết định (nới / siết / hiệu chuẩn từng máy). Và bạn có một chuỗi tư thế đồ gá đã được kiểm rằng nó kích thích đủ mọi tham số cần ước lượng. Hai sản phẩm ấy là hai thứ đắt nhất để làm lại nếu bỏ qua, và chúng tốn bốn ngày trên máy tính.
Nối với	C/14-hieu-chuan-nhu-mot-nhanh-cua-slam cho phần lý thuyết của việc kiểm chuỗi tư thế; C/18-danh-gia-va-phuong-phap-luan cho phương pháp Monte-Carlo có kỷ luật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Quyết định quan trọng nhất là *dung sai cơ khí*, và nó được trả lời bằng phân tích độ nhạy: nhiều loạn từng tham số trong mô phỏng theo độ tán dự kiến, đo sai số đầu ra, rồi siết dung sai cho các tham số nhạy và nới cho các tham số không nhạy. Nó phải đưa ra trước khi chốt cơ khí vì siết dung sai sau khi mở khuôn là bất khả về mặt kinh tế — lựa chọn còn lại là hiệu chuẩn từng máy suốt đời sản phẩm, vốn đắt hơn nhiều. Chiều *nới* cũng quan trọng và hay bị bỏ: phần lớn dự án siết mọi dung sai vì không biết cái nào quan trọng, và điều đó đắt một cách không cần thiết.
- A2. Đại lượng quyết định là tỉ số (*độ tán của tham số trong lô*) / (*mức tham số được phép sai theo ngân sách*). Từ số đo bằng cách hiệu chuẩn một mẫu vài chục máy và tính độ lệch chuẩn; mẫu số đến từ phân tích độ nhạy. Tỉ số nhỏ hơn một thì dùng chung được. Quyết định không cần đồng nhất cho mọi tham số: thường thì tiêu cự và méo khá đồng đều trong một lô module cùng nguồn nên dùng chung được, còn *ngoại* phụ thuộc lắp ráp nên độ tán lớn hơn và phải đo từng máy. Quy trình hỗn hợp — dùng chung nội, đo riêng ngoại — thường là điểm tối ưu và nhanh hơn nhiều so với đo tất cả.
- A3. (1) Benchmark *không chứa các chế độ hồng ở D/19* — không có gương, ít người đi lại, không có bụi ống kính, vì chúng được thu trong điều kiện kiểm soát. (2) Benchmark *quá ngắn* — không bộ nào dài tám tiếng, nên các vấn đề về kích thước bản đồ và lão hoá ở C/13 không xuất hiện. (3) Benchmark *đo sai thứ* — chúng đo ATE trung bình trên các chuỗi hệ thống chạy được, trong khi sản phẩm cần tỉ lệ thất bại và hành vi ở đuôi. Kết luận: không bao giờ tin benchmark công khai là đủ; cần bộ dữ liệu nội bộ gồm chuỗi đại diện, chuỗi nghịch cảnh, và — có giá trị nhất — chuỗi từ hiện trường.
- A4. Ba thứ phải thiết kế từ trước: (1) *các đại lượng giám sát* đo nguyên nhân chứ không chỉ hậu quả; (2) *cơ chế log ba tầng* — tầng một luôn bật và rẻ, tầng hai là vòng đệm quay vòng chỉ ghi ra khi có sự kiện (đây là tầng quan trọng nhất vì nó cho dữ liệu về chính khoảnh khắc trước khi hồng), tầng ba là ảnh và chỉ khi được phép; (3) *tính tất định*, vì không có nó thì log tầng hai vô dụng. Có ba thứ đó thì vòng lặp fleet analytics chạy được: phân cụm sự kiện *theo dấu hiệu* chứ không theo mô tả của khách hàng, xếp hạng cụm theo tần suất nhân mức nghiêm trọng, rồi tái hiện cụm lớn nhất trong lab. Nó biến một trăm báo cáo mơ hồ thành ba nguyên nhân xếp hạng.
- A5. Ba bước: liệt kê gauge cần phá (A/05); với mỗi ứng viên xem nó phá gauge nào và đòi điều kiện gì (C/15); kiểm tính *độc lập của chế độ hồng ở D/19*. Với robot dịch vụ trong nhà, kết luận của tôi là *stereo + IMU + odometry bánh xe, với global shutter*: stereo phá gauge tỉ lệ ngay từ khung đầu và xoá cả một lớp chế độ hồng khởi tạo; IMU ghim roll-pitch và sống khi thị giác chết; odometry cho tỉ lệ độc lập, ràng buộc nonholonomic miễn phí, và sống khi cả hai kia kém. Ba chế độ hồng gần như độc lập nên chúng kiểm chéo lẫn nhau. Điều kiện: kết luận này dành cho robot mặt

đất trong nhà, hoạt động không giám sát, có ngân sách cho ba cảm biến — nó không áp dụng cho drone, thiết bị cầm tay, hay sản phẩm cực rẻ.

NÓI LẠI THẬT ĐƠN GIẢN

Nấu một bữa ăn ngon và mở một nhà hàng là hai việc khác nhau về bản chất, không chỉ về quy mô. Nấu ở nhà: bạn nêm, bạn chỉnh, bạn dùng nguyên liệu mình thích. Kết quả phụ thuộc vào tay nghề của bạn hôm đó.

Mở nhà hàng: món phải giống nhau ở mọi bàn, mọi ngày, do nhiều người nấu, với nguyên liệu từ nhiều nguồn. Bạn không thể nêm từng đĩa.

Từ đó ra bốn việc mà người nấu ăn ở nhà không bao giờ nghĩ tới.

Công thức phải đo được, không phải “nêm vừa ăn” — vì người nấu không phải bạn và “vừa ăn” của họ khác của bạn.

Phải biết món nào hồng mà không cần nêm — nhìn màu, đo nhiệt độ bếp, canh thời gian. Các dấu hiệu gián tiếp thay cho việc nêm.

Phải biết khách nghĩ gì một cách có hệ thống — không phải từng lời phàn nàn lẻ tẻ mà là: món nào bị trả lại nhiều nhất, vào giờ nào, ở bàn nào. Một trăm lời phàn nàn mơ hồ gộp lại có thể chỉ ra đúng một cái bếp bị lệch nhiệt.

Và việc thứ tư, quan trọng nhất và dễ quên nhất: *phải quyết kích thước bếp trước khi xây nhà hàng*. Sau khi xây xong, muốn thêm một cái lò thì phải đập tường.

Trong một sản phẩm có camera, những quyết định kiểu “đập tường” là: hai camera cách nhau bao xa và được phép lệch bao nhiêu độ; dùng loại cảm biến ảnh nào; có gắn một cái đo nhiệt độ bên cạnh ống kính hay không. Ba thứ ấy phải quyết vào lúc chưa ai thấy chúng quan trọng — và đó chính là lý do chúng hay bị bỏ qua.

Cách duy nhất biết trước là *thử trên máy tính*: giả sử camera lệch nửa độ thì kết quả sai bao nhiêu? Nếu sai nhiều thì phải làm cơ khí chính xác hơn, hoặc phải đo riêng từng máy. Nếu sai ít thì có thể làm lỏng tay và tiết kiệm. Bốn ngày ngồi máy tính đổi lấy việc không phải mở lại khuôn nhựa — đó là phép tính dễ nhất trong cả chương.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *phải quyết định khi chưa có dữ liệu*. Bạn cần biết độ tán của tham số trong lô để quyết dung sai, nhưng bạn chỉ đo được độ tán sau khi đã sản xuất một lô — tức sau khi đã chốt dung sai. Cách duy nhất phá vòng là làm một lô mẫu nhỏ trước, và đó là một khoản chi phải thuyết phục được người trả tiền vào lúc chưa ai thấy nó cần.

Một câu: *Khoảng cách từ một máy chạy được tới nghìn máy chạy được nằm ở những quyết định phải đưa ra trước khi ai thấy chúng quan trọng.*

3.6 Điều gì chứng minh cách hiểu này sai

Toàn bộ chương này là *cấu trúc của các quyết định* chứ không phải các sự kiện đo được, nên nó bị bác bỏ theo một cách khác với các chương trước: không phải bằng một phép đo mà bằng một kinh nghiệm triển khai mâu thuẫn.

Mệnh đề chính — *phân tích độ nhạy phải chạy trước khi chốt cơ khí* — bị bác bỏ nếu trong thực tế, việc hiệu chuẩn từng máy rẻ tới mức không đáng siết dung sai cho bất kỳ tham số nào. Điều đó *có thể* đúng với sản phẩm sản lượng thấp giá cao, nơi ba phút dây chuyền không đáng kể. Nên phát biểu chặt hơn: mệnh đề đúng khi thời gian dây chuyền là ràng buộc, và mất hiệu lực khi không.

Mệnh đề thứ hai — *stereo + IMU + odometry là điểm ngọt cho robot mặt đất* — là suy luận của tôi từ ba chương trước, *không phải kết quả của một khảo sát hay một thử nghiệm so sánh*. Nó bị bác bỏ nếu một sản phẩm thành công trên thị trường dùng cấu hình rẻ hơn — một camera mono cộng IMU chẳng hạn — với tỉ lệ hỏng chấp nhận được. Tôi ngờ rằng điều này *có* xảy ra ở phân khúc giá rẻ, và nếu vậy thì kết luận đúng là “điểm ngọt phụ thuộc vào tỉ lệ hỏng mà thị trường chấp nhận”, không phải “ba cảm biến là câu trả lời”. Đây là mệnh đề tôi ít chắc nhất trong chương.

Mệnh đề thứ ba — *fleet analytics là vòng lặp quan trọng nhất* — bị bác bỏ nếu các chế độ hồng thật đủ đơn giản để tái hiện trong lab mà không cần dữ liệu hiện trường. Với một sản phẩm hoạt động trong môi trường kiểm soát — một nhà máy có điều kiện ổn định — điều đó có thể đúng. Với sản phẩm tiêu dùng thì tôi cho là không.

Và một cảnh báo bao trùm: chương này được viết *không có kinh nghiệm vận hành fleet thật*. Nó suy ra từ nguyên lý và từ các chương trước. Ai đã vận hành một đội thiết bị thật sẽ biết những thứ không có ở đây, và nếu bạn trở thành người đó thì hãy sửa chương này bằng kinh nghiệm của mình.

3.7 Kết nối

Chương này là chỗ hội tụ của gần như mọi chương trong cây, và cách đọc đúng là đọc nó như một bản đồ ngược.

Nối lên trên theo từng mục. Mục 2 (hiệu chuẩn) lấy từ A/02 (mô hình và chẩn đoán), A/05 (điều kiện kích thích), A/06 (hiệp phương sai tham số), và C/14 (khung hợp nhất) — và biến chúng thành một quy trình có chỉ tiêu. Mục 3 (độ tin cậy) lấy từ D/19 (chế độ hồng và phát hiện) và D/20 (logging và tính tắt định). Mục 4 (kiểm thử) lấy từ C/18 (phương pháp đánh giá) và D/19 (bộ nghịch cảnh). Mục 5 (ràng buộc sản phẩm) lấy từ A/05 và C/15 (chọn cảm biến theo gauge), A/02 (global shutter), C/13 (quyền riêng tư và định dạng bản đồ), D/20 (nhiệt).

Nối xuống dưới: D/22-huong-nghien-cuu-con-trong là chương cuối, và ba trong tám hướng ở đó — hiệu chuẩn có ý thức observability, dự đoán thất bại, hệ tự chữa — đều mọc ra từ các vấn đề nêu trong chương này. Đó là một ví dụ của điều mà `research-rules.md` mục 7 nói: câu hỏi nghiên cứu tốt mọc ra từ một vấn đề thật, không từ cảm hứng ngẫu nhiên.

Nối ngang: `../marker/` là cây bàn về một giải pháp thực dụng cho bài toán định vị tuyệt đối trong môi trường có thể sửa đổi, và nó có một phần riêng về ngân sách sai số và nghiệm thu — rất gần tinh thần chương này, ở một quy mô nhỏ hơn và cụ thể hơn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu bốn quyết định không sửa được sau khi chốt cơ khí, và nói mỗi cái được quyết bởi phân tích nào.
2. Trình bày quy trình phân tích độ nhảy bốn bước, và nói vì sao chiều “nói dung sai” cũng quan trọng.
3. Nêu tỉ số quyết định việc hiệu chuẩn từng máy hay dùng chung, và nói vì sao quyết định có thể khác nhau giữa tham số nội và tham số ngoại.
4. Vì sao chuỗi tư thế của đồ gá nên được kiểm bằng phân tích observability trước khi chế tạo?
5. Nêu ba tầng logging và nói tầng nào quan trọng nhất cho việc chẩn đoán, cùng điều kiện để nó có giá trị.
6. Mô tả vòng lặp fleet analytics bốn bước, và nói ba thứ phải thiết kế từ trước để nó khởi động được.
7. Trình bày ba bước biến việc chọn cảm biến thành một bài toán có tiêu chí, và nêu kết luận cùng *điều kiện* của nó.

Nguồn

- [1] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [2] J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [6] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [7] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [8] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [9] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.
- [10] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [13] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.
- [14] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.

- [15] J. Maye, P. Furgale, and R. Siegwart, "Self-Supervised Calibration for Robotic Systems," in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [16] T. Schneider, M. Li, C. Cadena, et al., "Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation," in *IEEE Sensors Journal*, mức tin B, 2019.

Tám hướng còn khoảng trống (Eight Open Directions)

PHỤ THUỘC Toàn bộ cây. Mỗi hướng dưới đây mọc ra từ một vấn đề cụ thể đã nêu ở một chương trước, và phần “từ đâu ra” của mỗi mục trở về chương đó.

TÓM TẮT

Chương này là chương duy nhất trong cây chạm vào địa hạt nghiên cứu, và nó mở đầu bằng một cảnh báo về chính nó: *nó chưa đạt chuẩn của research-rules.md*. Nó không dựa trên một khảo sát có hệ thống — không có nhật ký tìm kiếm, không có ma trận tổng hợp, không đạt chỉ tiêu số công trình cho bất kỳ hướng nào. Nó là danh sách các chỗ mà *tôi cho là còn trống*, dựa trên việc đọc rải rác và trên lập luận cấu trúc từ hai mươi mốt chương trước. Vì vậy mỗi hướng được trình bày theo một khuôn cố định gồm bốn phần: *từ đâu ra* (vấn đề cụ thể ở chương nào), *vì sao tôi cho là còn trống*, *mức chắc chắn của tôi*, và — phần quan trọng nhất — *phép kiểm* để xác minh nó có thật là khoảng trống hay chỉ là chỗ tôi chưa đọc tới. Tám hướng, xếp theo mức tôi tin là chúng còn trống và đáng làm. Nếu chỉ nhớ một điều: *trước khi chọn bất kỳ hướng nào, hãy chạy phép kiểm của nó* — vì cách nhanh nhất để lãng phí một năm là phát minh lại thứ người ta đã giải quyết ba năm trước.

CẠM BÃY

Cảnh báo về chính chương này, và nó là phần quan trọng nhất cần đọc.

Theo *research-rules.md*, một khảo sát đạt chuẩn cần: nhật ký tìm kiếm tái lập được; sàng ít nhất 25–40 công trình cho mỗi chủ đề; ít nhất ba nhóm nghiên cứu độc lập; ma trận tổng hợp viết theo cột; và mọi khẳng định lõi tựa vào một nguồn mức A hoặc hai nguồn mức B độc lập.

Chương này không đạt bất kỳ điều kiện nào trong số đó. Nó được viết từ lập luận cấu trúc và từ việc đọc rải rác, không từ một khảo sát. Điều đó nghĩa là:

Rủi ro chính: một “khoảng trống” ở đây có thể đã được lấp bởi các công trình tôi chưa đọc. Đây là rủi ro có thật và không nhỏ, đặc biệt với các hướng liên quan tới học máy, nơi tài liệu tăng rất nhanh.

Cách dùng đúng: coi mỗi mục dưới đây là một **giả thuyết cần kiểm**, không phải một kết luận. Mỗi mục kèm một phép kiểm cụ thể, và phép kiểm ấy nên được chạy *trước* khi đầu tư thời gian vào hướng đó.

Việc cần làm nếu dùng chương này để chọn đề tài thật: nâng hướng đã chọn lên thành một dự án survey/ đúng chuẩn — bước 0 tới bước 4 của *research-rules.md* — rồi mới quyết. Chương này chỉ là bước chọn *ứng viên*, không phải bước kết luận.

4.1 Vì sao một danh sách như thế này lại có giá trị

Nếu chương này không đạt chuẩn khảo sát, vì sao viết nó?

Vì nó làm một việc mà một khảo sát không làm: nó *nối các vấn đề đã gặp trong cây thành các câu hỏi*. Hai mươi mốt chương trước liên tục chạm vào các chỗ mà tôi phải viết “chưa có lời giải tốt” —

bài toán lifelong ở C/13, bệnh quá tự tin ở A/06, perceptual aliasing ở B/11. Những chỗ ấy rải rác và dễ quên; gom chúng lại là một việc có giá trị riêng.

Theo research-rules.md mục 7, một câu hỏi nghiên cứu tốt mọc ra từ một vấn đề thật chứ không từ cảm hứng ngẫu nhiên. Tám mục dưới đây đều mọc ra từ một vấn đề đã gặp, và phần “từ đâu ra” của mỗi mục là bằng chứng cho điều đó.

Tiêu chí xếp thứ tự: *mức tôi tin là nó còn trống*, nhân với *mức tôi tin là nó đáng giải*. Hai yếu tố ấy độc lập — một bài toán có thể còn trống vì nó không đáng giải.

4.2 Hướng 1 — Hiệu chuẩn trực tuyến có ý thức observability

Từ đâu ra: C/14 mục 3. Hiệu chuẩn trực tuyến cần một cổng quyết định khi nào cập nhật tham số nào, và các cơ chế hiện có còn thô sơ: hoặc cập nhật mù quáng, hoặc dùng các quy tắc thủ công dựa trên một đại lượng chuyển động duy nhất.

Bài toán cụ thể: cho một hệ đang chạy, quyết định *theo thời gian thực* tham số nào đang quan sát được đủ để cập nhật. Cần một chỉ số rõ, một ngưỡng có cơ sở, và một cơ chế chuyển đổi mượt giữa cập nhật và đóng băng.

Vì sao tôi cho là còn trống: các công trình tôi biết [15], [16] thiết lập ý tưởng nhưng dừng ở mức phân tích phổ, vốn đắt cho thời gian thực. Phần lớn hệ mã nguồn mở hoặc không có cổng hoặc dùng ngưỡng thủ công.

Mức chắc chắn: trung bình. Ý tưởng đã có; điều tôi cho là còn trống là một cơ chế *thực dụng và được kiểm chứng*.

Phép kiểm: tìm các công trình sau 2020 với từ khoá “observability-aware calibration”, “degeneracy-aware state estimation”, “online extrinsic gating”. Kiểm xem có công trình nào (a) chạy thời gian thực, (b) có ngưỡng suy ra từ nguyên lý chứ không chỉnh tay, và (c) được đánh giá bằng *tính nhất quán* chứ chỉ bằng độ chính xác. Nếu cả ba đã có, hướng này đã được lấp.

4.3 Hướng 2 — Bất định đáng tin

Từ đâu ra: A/06 mục 4 và C/17 mục 4. SLAM quá tự tin một cách hệ thống, và bốn nguồn của nó đều khó chữa bằng phân tích.

Bài toán cụ thể: một hiệp phương sai mà module phía trên *dùng được* — tức có ANEES gần số chiều trên dữ liệu thật, không chỉ trong mô phỏng, và trong các điều kiện khó chứ không chỉ điều kiện tốt.

Vì sao tôi cho là còn trống: hai lý do cấu trúc đã nêu ở C/17 mục 4A. Nó khó đo (cần chân trị và thống kê), nên không có chỉ số dễ công bố; và một bài cải thiện ATE dễ đăng hơn một bài cải thiện tính trung thực.

Mức chắc chắn: **cao** — đây là hướng tôi tin nhất trong tám hướng. Lý do: tôi có thể chỉ ra bốn cơ chế cụ thể gây quá tự tin (A/06 mục 4B) và không cơ chế nào có lời giải sạch.

Phép kiểm: khảo sát xem có bao nhiêu công trình SLAM báo cáo NEES hoặc ANEES bên cạnh ATE. Nếu tỉ lệ rất thấp thì hướng còn trống được xác nhận theo một chiều — ít người đo, nên ít người cải thiện. Bổ sung: tìm các công trình về “uncertainty calibration” trong học máy và kiểm xem chúng đã được áp dụng cho SLAM chưa.

Ghi chú: hướng này có một ưu điểm phương pháp đáng kể — nó có một chỉ số rõ ràng (ANEES) và một tiêu chí thành công rõ ràng (gần số chiều). Nhiều hướng khác trong danh sách này thiếu điều đó.

4.4 Hướng 3 – Bản đồ sống lâu trong thế giới thay đổi

Từ đâu ra: C/13 mục 4C và B/12 mục 4C. Gần như mọi hệ SLAM giả định thế giới tĩnh ở thang thời gian dài, và đó là giả định sai nhất trong triển khai thực tế.

Bài toán cụ thể: phân biệt “bản đồ sai” với “tôi ở sai chỗ” — hai kết luận cho cùng triệu chứng và đòi hai hành động ngược nhau. Và cái vòng luẩn quẩn: để biết bản đồ sai, phải chắc mình ở đúng chỗ; để chắc mình ở đúng chỗ, phải tin bản đồ.

Vì sao tôi cho là còn trống: các cách tiếp cận hiện có đều là cách phá vòng một cách thô — quên theo thời gian, đếm phiếu qua nhiều lần đi qua, lưu nhiều phiên bản — và không cách nào giải quyết trường hợp thay đổi cấu trúc lớn.

Mức chắc chắn: cao về việc nó là vấn đề đau nhất trong triển khai; trung bình về việc nó còn trống, vì tôi biết có một nhánh công trình về lifelong SLAM mà tôi chưa khảo sát.

Phép kiểm: đây là hướng có phép kiểm rõ ràng nhất vì nó có một chỉ tiêu tự nhiên. Tìm xem có hệ nào duy trì được tỉ lệ *relocalization* ổn định qua nhiều tháng trong một môi trường thay đổi thật, được kiểm định độc lập. Bộ dữ liệu để kiểm: OpenLORIS [4] và các bộ nhiều phiên tương tự. Nếu không hệ nào làm được, hướng còn trống.

Ghi chú: hướng này cần dữ liệu dài hạn, vốn khó thu — và đó vừa là rào cản vừa là lý do nó còn trống. Mini project ở C/13 là một cách bắt đầu thu.

4.5 Hướng 4 – Dự đoán thất bại thay vì phục hồi sau

Từ đâu ra: D/19 mục 6. Mọi cơ chế hiện có là *phản ứng*: phát hiện sau khi đã mất bám, rồi phục hồi.

Bài toán cụ thể: dự đoán trước vài giây rằng hệ thống sắp mất bám, để module phía trên có thể hành động — robot đi chậm lại, quay về vùng đã biết, hoặc chuyển chế độ trước khi hỏng.

Vì sao tôi cho là còn trống: các đại lượng cần thiết đã có sẵn (D/19 khối *Cần nhớ*) và chúng thay đổi trước khi hỏng — số đặc trưng giảm, số điều kiện tăng. Việc nối chúng thành một bộ dự đoán là một bài toán chuỗi thời gian thông thường. Tôi không biết công trình nào làm điều này một cách hệ thống trong SLAM.

Mức chắc chắn: trung bình. Ý tưởng đủ hiển nhiên khiến tôi ngờ rằng ai đó đã làm, và việc tôi không biết có thể chỉ phản ánh việc tôi chưa tìm.

Phép kiểm: tìm với từ khoá “failure prediction SLAM”, “predictive tracking failure”, “proactive relocalization”. Nếu có công trình, kiểm xem chúng dự đoán trước bao lâu và với tỉ lệ báo động giả nào — vì một bộ dự đoán báo trước 0,2 giây thì vô dụng cho việc ra quyết định.

Ghi chú: hướng này có lợi thế là *dữ liệu dễ có* — mọi lần mất bám trong bất kỳ chuỗi nào đều là một mẫu, và bộ nghịch cảnh ở D/19 mini project sinh ra chúng có chủ đích.

4.6 Hướng 5 – Bất định được truyền đúng qua ranh giới học/hình học

Từ đâu ra: C/17 mục 4B và B/08 mục 4C. Kiến trúc lai là hướng đúng, nhưng phần lớn công trình nổi hai khối một cách lỏng lẻo: mô hình học được cho một phép đo, và hiệp phương sai của phép đo ấy được đặt bằng một hằng số chính tay.

Bài toán cụ thể: một mô hình học được sinh ra phép đo kèm hiệp phương sai đã hiệu chuẩn, và hiệp phương sai ấy đúng cả khi dữ liệu nằm ngoài phân bố huấn luyện.

Vì sao tôi cho là còn trống: về cuối là phần khó và là phần tôi cho là chưa giải. Một mô hình biết mình không chắc khi gặp cảnh lạ đòi hỏi phát hiện ngoài phân bố, và đó là bài toán mở trong chính học máy.

Mức chắc chắn: trung bình-cao. Cao về việc bài toán chưa giải; trung bình về việc nó là một khoảng trống riêng của SLAM — có thể nó chỉ là bài toán chung của học máy được nhìn qua lăng kính SLAM.

Phép kiểm: hai bước. Một, tìm trong tài liệu học máy các công trình về hiệu chuẩn bất định và phát hiện ngoài phân bố sau 2023 — nếu có tiến bộ lớn thì hướng này trở thành việc áp dụng chứ không phải nghiên cứu. Hai, kiểm xem có công trình SLAM nào dùng hiệp phương sai học được trong đồ thị nhân tử và đo ANEES của kết quả.

Ghi chú: hướng này gắn chặt với hướng 2, và chúng có thể là một — hoặc nên được làm cùng nhau.

4.7 Hướng 6 – Xử lý suy biến có nguyên tắc

Từ đâu ra: A/05 mục 2C và A/01 mục 6. Việc phát hiện và xử lý chuyển động suy biến hiện chủ yếu là heuristic: các ngưỡng thủ công cho parallax, cho tỉ số điểm số mô hình, cho số điều kiện.

Bài toán cụ thể: một khung thống nhất để (a) phát hiện hướng suy biến theo thời gian thực với chi phí chấp nhận được, (b) phân biệt ba loại suy biến ở A/05 mục 2C, và (c) quyết định hành động — đóng băng biến nào, dùng mô hình nào, hay từ chối.

Vì sao tôi cho là còn trống: mỗi hệ thống tự xử lý theo cách riêng với các ngưỡng riêng, và không có một khung chung. Đặc biệt phần (b) — phân biệt ba loại — tôi không biết cài đặt nào làm một cách tường minh.

Mức chắc chắn: trung bình. Có nhiều công trình về observability phân tích; điều tôi cho là còn trống là một khung chạy được và thống nhất.

Phép kiểm: kiểm xem các thư viện lớn (GTSAM, Ceres, g2o) có cung cấp một cơ chế báo hướng suy biến không, và các hệ SLAM lớn có dùng nó không. Nếu mỗi hệ vẫn tự cài ngưỡng riêng thì khoảng trống được xác nhận. Bổ sung: tìm “degeneracy detection” trong tài liệu LiDAR SLAM, nơi vấn đề này (hành lang dài) gay gắt hơn và có thể đã được giải tốt hơn.

4.8 Hướng 7 – Hệ tự nhận biết lệch và tự sửa

Từ đâu ra: C/14 mục 4B và D/21 mục 2D. Một thiết bị ở hiện trường phải tự biết hiệu chuẩn của nó đã trôi và tự xử lý, vì không ai mang target tới nhà khách hàng.

Bài toán cụ thể: một vòng khép kín gồm phát hiện trôi (không có target, không có chân trị), phân biệt nguyên nhân (nhiệt so với va đập so với lão hoá), tự sửa khi có thể, và báo khi không.

Vì sao tôi cho là còn trống: các mảnh đã có — hiệu chuẩn trực tuyến, phân tích phần dư — nhưng tôi không biết một hệ nào ghép chúng thành một vòng khép kín có đánh giá.

Mức chắc chắn: trung bình-thấp. Đây có thể là một vấn đề kỹ thuật đã được các công ty giải quyết mà không công bố, chứ không phải một khoảng trống nghiên cứu.

Phép kiểm: khác với các hướng khác, phép kiểm ở đây không phải tìm trong tài liệu học thuật mà là tìm trong tài liệu kỹ thuật của sản phẩm — các thiết bị AR và robot thương mại có công bố cách xử lý trôi hiệu chuẩn không. Nếu có thì đây là vấn đề đã giải nhưng không công bố, và giá trị nghiên cứu thấp hơn giá trị kỹ thuật.

4.9 Hướng 8 — Biểu diễn neural cho localization thực dụng

Từ đâu ra: C/16 mục 4B. 3DGS cho hình ảnh đẹp nhưng ba điểm chặn cho robot: không trả lời được truy vấn occupancy, tổn bộ nhớ và cần GPU, và chế độ hồng chưa được hiểu.

Bài toán cụ thể: một biểu diễn neural mà từ đó trích xuất được ESDF tin cậy, chạy trong ngân sách của một SoC nhúng, và có chế độ hồng dự đoán được.

Vì sao tôi cho là còn trống: phần lớn công trình tối ưu cho chất lượng render, không cho truy vấn hình học. Ba điểm chặn ở C/16 mục 4B đều còn.

Mức chắc chắn: **thấp** — đây là hướng tôi ít chắc nhất, vì lĩnh vực chuyển động rất nhanh và đánh giá của tôi chốt tại 9/2026. Rất có thể một hoặc cả ba điểm chặn đã được giải trong thời gian tôi viết chương này.

Phép kiểm: rà lại tài liệu 3DGS-SLAM sau mỗi sáu tháng, và kiểm cụ thể ba điểm: có công trình nào trích xuất occupancy hoặc ESDF từ Gaussian với đánh giá định lượng không; có công trình nào chạy trên phần cứng nhúng không; có công trình nào phân tích chế độ hồng không.

CẦN NHỚ

Cách dùng danh sách này, theo thứ tự.

Một — chạy phép kiểm trước. Mỗi mục có một phép kiểm cụ thể. Chạy nó mất vài ngày và nó có thể tiết kiệm một năm. Cách nhanh nhất để lãng phí thời gian là phát minh lại thứ đã được giải quyết.

Hai — nếu phép kiểm xác nhận khoảng trống, nâng lên thành khảo sát đúng chuẩn. research-rules.md bước 0 tới bước 4: dựng khung câu hỏi, săn nguồn theo giao thức lần cầu tuyết, sàng theo mức tin A/B/C, đọc ba lượt, lập ma trận tổng hợp. Chỉ sau bước đó mới đủ cơ sở để chọn.

Ba — ưu tiên các hướng có chỉ số rõ ràng. Hướng 2 (ANEES gần số chiều) và hướng 3 (tỉ lệ relocalization ổn định qua tháng) có tiêu chí thành công đo được. Các hướng thiếu điều đó khó chứng minh tiến bộ, và một bài toán không đo được là một bài toán không giải được.

Bốn — ưu tiên các hướng mà bạn có dữ liệu. Hướng 3 và 4 cần dữ liệu dài hạn và dữ liệu thất bại; nếu bạn đang vận hành thiết bị thật thì bạn có lợi thế mà phòng thí nghiệm không có — và đó là lợi thế đáng khai thác hơn là cố cạnh tranh ở chỗ mọi người đều mạnh.

Và một gợi ý cuối, thuộc về E/24: một bài toán ứng dụng cụ thể mà bạn đang làm thường là chỗ tốt hơn để bắt đầu so với một hướng trong danh sách này — vì nó có ràng buộc thật, có dữ liệu thật, và có tiêu chí thành công thật.

4.10 Điều gì chứng minh cách hiểu này sai

Chương này khác các chương khác: mệnh đề của nó không phải một khẳng định kỹ thuật mà là *tám phán đoán về trạng thái của lĩnh vực*, và mỗi phán đoán đã kèm mức chắc chắn cùng phép kiểm riêng ở mục tương ứng.

Mệnh đề bao trùm — *tám hướng này còn trống* — bị bác bỏ từng phần bởi chính các phép kiểm ấy. Tôi kỳ vọng ít nhất hai trong tám sẽ bị bác bỏ khi kiểm nghiệm túc, và hai ứng viên có khả năng nhất là hướng 4 (dự đoán thất bại — ý tưởng đủ hiển nhiên) và hướng 8 (3DGS — lĩnh vực chuyển động quá nhanh).

Có một mệnh đề thứ hai, ẩn hơn và đáng nêu: *rằng một danh sách như thế này có giá trị*. Nó bị bác bỏ nếu việc chọn đề tài nghiên cứu từ một danh sách viết sẵn cho kết quả tệ hơn việc chọn từ một vấn đề bạn đang thật sự gặp. Tôi *ngghiêng về* khả năng đó — và tôi đã nói điều đó ở cuối khối *Cần nhớ*. Danh sách này hữu ích nhất không phải như một thực đơn để chọn mà như một *bản đồ để nhận ra* rằng vấn đề bạn đang gặp trùng với một chỗ mà lĩnh vực cũng chưa giải.

Và một cảnh báo cuối, nhắc lại từ đầu chương vì nó quan trọng: chương này được viết *không có khảo sát*. Nếu bạn dùng nó để chọn đề tài mà không chạy phép kiểm, bạn đang dựa vào phán đoán của một người đọc rải rác — và theo chính `research-rules.md` mục 8, đó là đúng thứ không nên tin.

4.11 Kết nối

Chương này nói ngược về gần như toàn bộ cây, và bảng dưới đây là bản đồ ấy.

Hướng 1 về C/14 và A/05. Hướng 2 về A/06 và C/18. Hướng 3 về C/13 và B/12. Hướng 4 về D/19. Hướng 5 về C/17 và B/08. Hướng 6 về A/05 và A/01. Hướng 7 về C/14 và D/21. Hướng 8 về C/16.

Nối xuống dưới: E/24-lo-trình giai đoạn năm là chỗ chương này được dùng, và nó nêu rõ quy trình — chọn một hướng, đọc 20–30 công trình gần nhất *của riêng hướng đó*, tái hiện hai tới ba baseline, rồi mới đề xuất. Thứ tự ấy quan trọng: tái hiện baseline trước khi đề xuất là cách duy nhất biết mình đang đứng ở đâu.

Nối sang `research-rules.md`: chương này là bước *trước* bước 0 của tài liệu đó. Nó chọn ứng viên; `research-rules.md` bắt đầu từ việc dựng khung câu hỏi cho ứng viên đã chọn. Nhầm hai bước — coi chương này là kết quả của một khảo sát — là lỗi phương pháp nghiêm trọng nhất có thể mắc với nó.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chương này không đạt chuẩn `research-rules.md`, và điều đó đổi cách dùng nó thế nào?
2. Với mỗi hướng, phần nào của mục là quan trọng nhất và vì sao?
3. Hướng nào tôi chắc chắn nhất và hướng nào ít chắc nhất? Nêu lý do của cả hai.
4. Vì sao hướng 2 và hướng 3 được đề nghị ưu tiên, và tiêu chí chung của chúng là gì?
5. Hướng 7 có phép kiểm khác các hướng khác. Khác thế nào, và điều đó nói gì về bản chất của hướng ấy?
6. Nêu bốn bước dùng danh sách này, và nói bước nào tiết kiệm nhiều thời gian nhất.
7. Vì sao một vấn đề bạn đang thật sự gặp thường là chỗ tốt hơn để bắt đầu so với một hướng trong danh sách?

Nguồn

- [1] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [2] J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [6] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [7] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [8] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [9] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.
- [10] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [13] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.
- [14] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.

- [15] J. Maye, P. Furgale, and R. Siegwart, "Self-Supervised Calibration for Robotic Systems," in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [16] T. Schneider, M. Li, C. Cadena, et al., "Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation," in *IEEE Sensors Journal*, mức tin B, 2019.

Nguồn

- [1] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [2] J. Engel, V. Koltun, and D. Cremers, “Direct Sparse Odometry,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [6] N. Demmel, C. Sommer, D. Cremers, et al., “Square Root Bundle Adjustment for Large-Scale Reconstruction,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [7] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [8] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [9] S. Agarwal, K. Mierle, and The Ceres Solver Team, *Ceres Solver*, phần mềm mã nguồn mở, mức tin B; tài liệu phần mềm, không phải công trình phản biện, 2022.
- [10] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [13] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.
- [14] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.

- [15] J. Maye, P. Furgale, and R. Siegwart, "Self-Supervised Calibration for Robotic Systems," in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [16] T. Schneider, M. Li, C. Cadena, et al., "Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation," in *IEEE Sensors Journal*, mức tin B, 2019.